Ünite 1: Giriş

- Veritabanı Sistemlerinin Amacı
- Verilerin Görünümü
- Veri Modelleri
- Veri Tanımlama Dili(DDL)
- Veri İşleme Dili(DML)
- İşlem Yönetimi
- Depolama Yönetimi
- Veritabanı Yöneticisi
- Veritabanı Kullanıcıları
- Genel Sistem Yapısı

Veritabanı Yönetim Sistemi (DBMS)

- Birbiriyle ilişkili veriler topluluğu
- Verilere erişmek için program seti
- Verilerin kolayca ve verimli bir şekilde depolanması
- Veritabanının yönetilmesi(MySQL, MSSQL Server, Oracle)
- DBMS, belirli bir kuruluş hakkında bilgi içerir
- DBMS, kullanımı kolay ve verimli bir ortam sağlar.
- Veritabanı Uygulamaları :
  - Bankacılık: tüm işlemler
  - Havayolları: rezervasyonlar, planlamalar
  - Üniversiteler: kayıt, notlar
  - Satış: müşteriler, ürünler, satın almalar
  - İmalat: üretim, envanter, siparişler, tedarik zinciri
  - İnsan kaynakları: çalışan kayıtları, maaşları, vergi indirimleri
- Veritabanları hayatımızın her alanına dokunur

Veritabanı Sisteminin Amacı

- İlk zamanlarda veritabanı uygulamaları dosya sistemlerinin üzerine inşa ediliyordu.
- Verileri depolamak için dosya sistemleri kullanmanın sakıncaları:
  - Veri tekrarı ve tutarsızlığı
    - Birden çok dosya biçimi, bilgilerin farklı dosyalarda kopyalanması
  - Verilere erişimde zorluk
    - Her yeni görevi yerine getirmek için yeni bir program yazmanız gerekiyor
  - Veri izolasyonu - birden çok dosya ve format
  - Bütünlük sorunları
    - Bütünlük kısıtlamaları (ör. Hesap bakiyesi> 0) program kodunun parçası olur
    - Yeni kısıtlamalar eklemek veya mevcut olanları değiştirmek zor

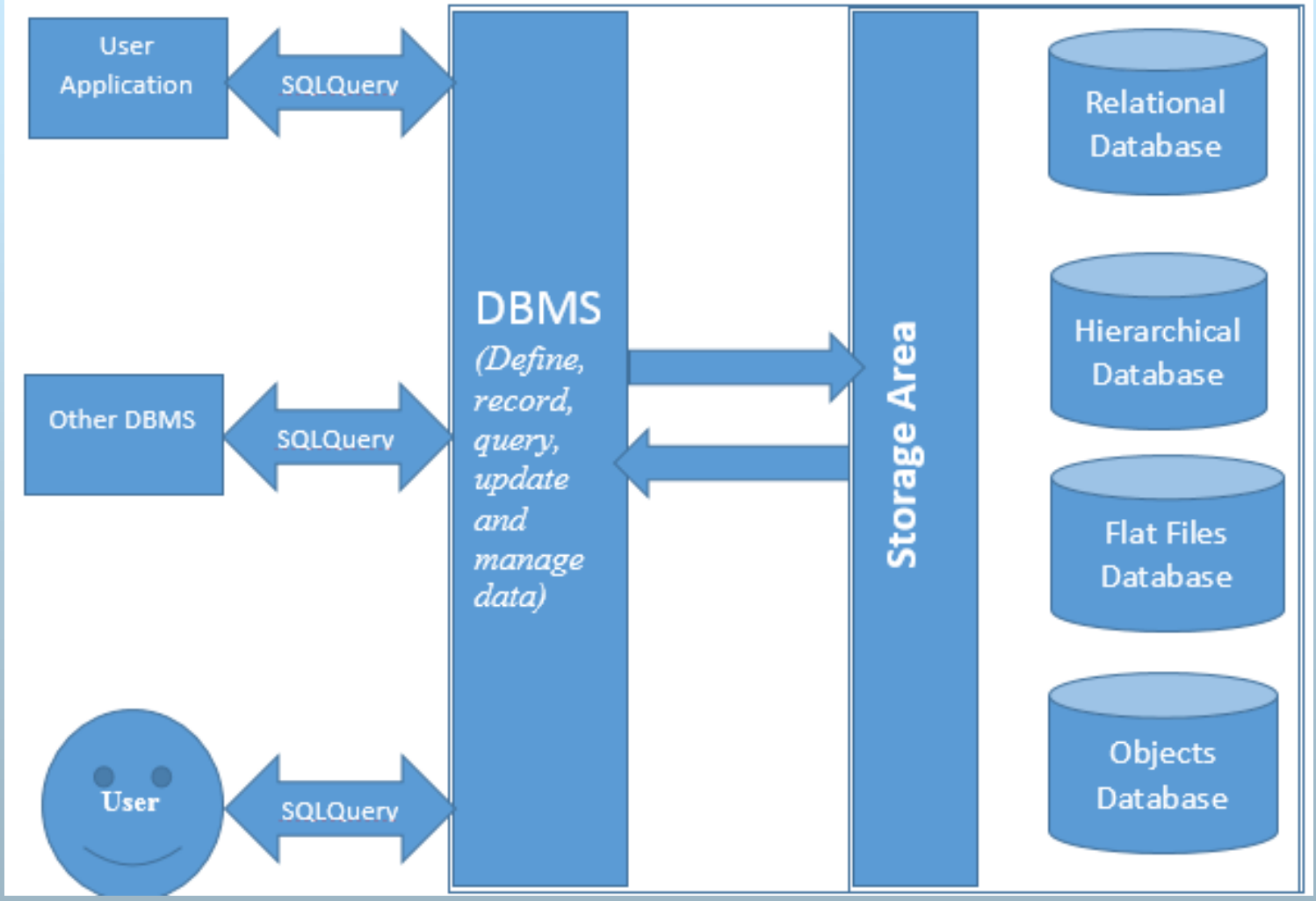
Veritabanı Sistemlerinin Amacı (Devam)

- Dosya sistemlerini kullanmanın sakıncaları (devam)
  - Güncellemelerin atomikliği
    - Hatalar, gerçekleştirilen kısmi güncellemelerle veritabanını tutarsız bir durumda bırakabilir
    - Örneğin. Bir hesaptan diğerine para transferi tamamlanmış olmalı veya hiç olmamalıdır
  - Birden çok kullanıcı tarafından eşzamanlı erişim
    - Performans için eşzamanlı erişim gerekli
    - Kontrolsüz eşzamanlı erişim tutarsızlıklara yol açabilir
      - Örneğin. iki kişi bir bakiyeyi okuyup aynı anda güncelliyor
  - Güvenlik sorunları
- Veritabanı sistemleri yukarıdaki tüm sorunlara çözümler sunar

Veritabanı Nedir?

- **Düzenli bilgiler topluluğudur.**
- Bilgisayar ortamında saklanan düzenli verilerdir.
- Bilgisayar terminolojisinde, sistematik erişim imkanı olan, yönetilebilir, güncellenebilir , taşınabilir, birbirleri arasında tanımlı ilişkiler bulunabilen bilgiler kümesidir.
- Bilgisayarda sistematik şekilde saklanmış, programlarca işlenebilecek veri yığınıdır

Veritabanı Nedir?



DBMS Bileşenleri

- Kullanıcılar
  - Son kullanıcılar, DB Yönetim
- Yazılımlar
  - Organizasyon
  - Depolama
  - Yönetim
  - Verilerin DB'den alınması
- Hardware
  - Disk…
- Data: Veritabanında depolanır
  - Numeric
  - Non numeric
  - Logical data

DBMS

- Veri yapı taşları
  - Kolonlar / Alanlar
  - Satır / tuples / kayıt
  - Tablolar
- Avantajları
  - Veri bağımsızlığı
  - Veriye etkili(verimli) erişim
  - Veri bütünlüğü ve güvenliği
  - Veri yönetimi
  - Eş zamanlı erişim
  - Uygulama süresinin kısaltılması

DBMS

- Veritabanı Niçin Kullanılmaz
  - DBMS kullanmanın genel maliyeti
    - Başlangıçta donanım, yazılım ve eğitime yüksek yatırım
    - veri tanımlama ve işleme maliyeti
    - güvenlik, eşzamanlılık kontrolü ve kurtarma için ek yük
- Bu nedenle, dosyaların düzenli olarak kullanılması daha cazip olabilir.
  - Değişmesi beklenmeyen basit, iyi tanımlanmış DB uygulaması.
  - verilere birden fazla kullanıcı erişiminin olmaması

DBMS – 3 Seviyeli Mimari

- Dış Seviye(External Level / View Level)
  - Bu düzey, DB'nin her bir kullanıcıyla ilgili olan kısmını tanımlar.
  - Verilerin görüntülenebildiği seviye
  - Kullanıcının veri yapısı, tablo tanımı vb. gibi veritabanı şeması ayrıntılarını bilmesine gerek yoktur.
  - Kullanıcı yalnızca, veritabanından getirildikten sonra görünüm düzeyine geri dönen verilerle ilgilenir
  - Böylece farklı kullanıcılar, veritabanını kendi gereksinimlerine göre farklı bir görünüm olarak görebilir.
- Kavramsal Seviye(Conceptual Level / Logical Level)
  - Bu seviye, veritabanında hangi verilerin depolandığını ve veriler arasındaki ilişkiyi tanımlar
    - tüm varlıklar, özellikler ve bunların ilişkileri
    - veriler üzerindeki kısıtlamalar
    - güvenlik ve bütünlük bilgileri

DBMS – 3 Seviyeli Mimari(Devam)

- İç Seviye(Internal Level / Storage Level / Physical Level)
  - Bu seviye aynı zamanda fiziksel seviye olarak da bilinir.
  - Bu seviye, verilerin depolama cihazlarında gerçekte nasıl depolandığını açıklar.
  - Bu seviye aynı zamanda verilere alan tahsis etmekten de sorumludur.
  - Bu, mimarinin en düşük seviyesidir.
  - Fiziksel düzey ayrıca sıkıştırma ve şifreleme tekniklerini de belirler.

Soyutlama Düzeyleri

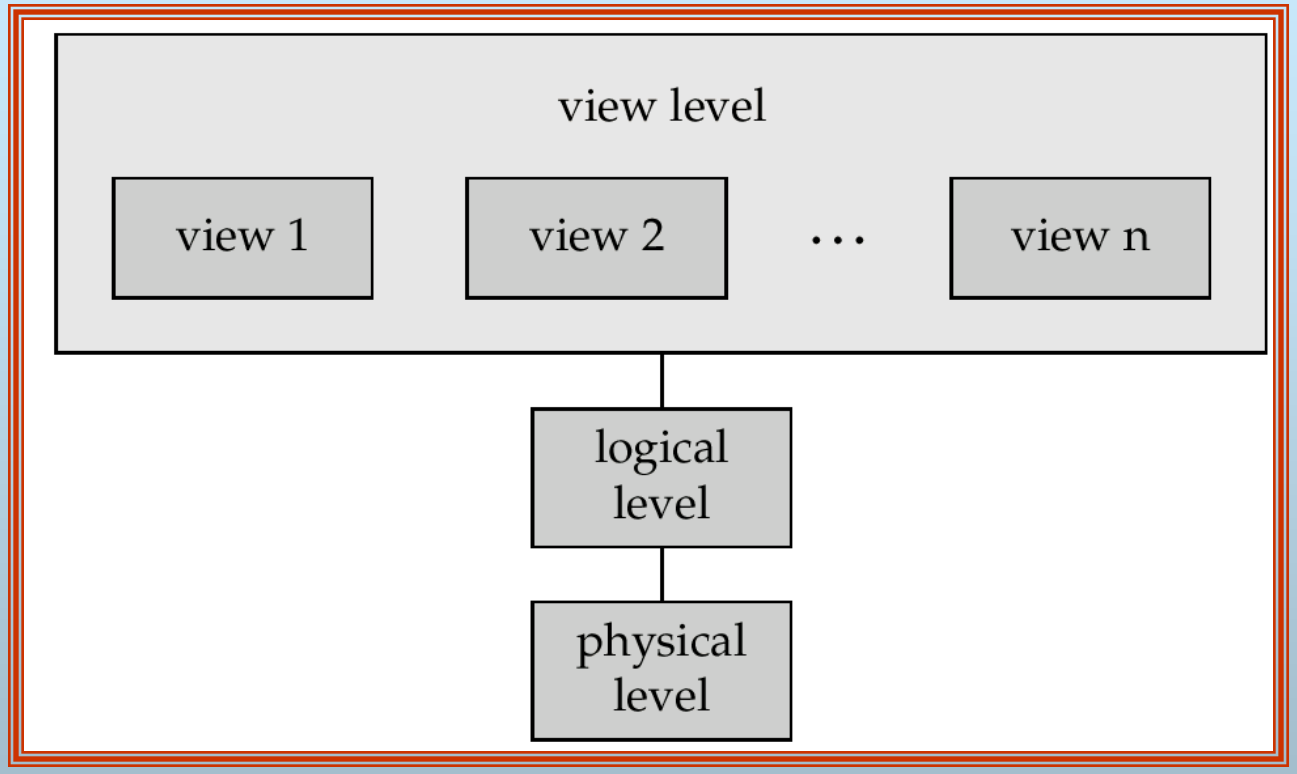
- Fiziksel seviye, bir kaydın (örneğin, müşteri) nasıl saklandığını açıklar.
- Mantıksal düzey: veritabanında depolanan verileri ve veriler arasındaki ilişkileri açıklar.

```
type customer = record
    name : string;
    street : string;
    city : integer;
end;
```

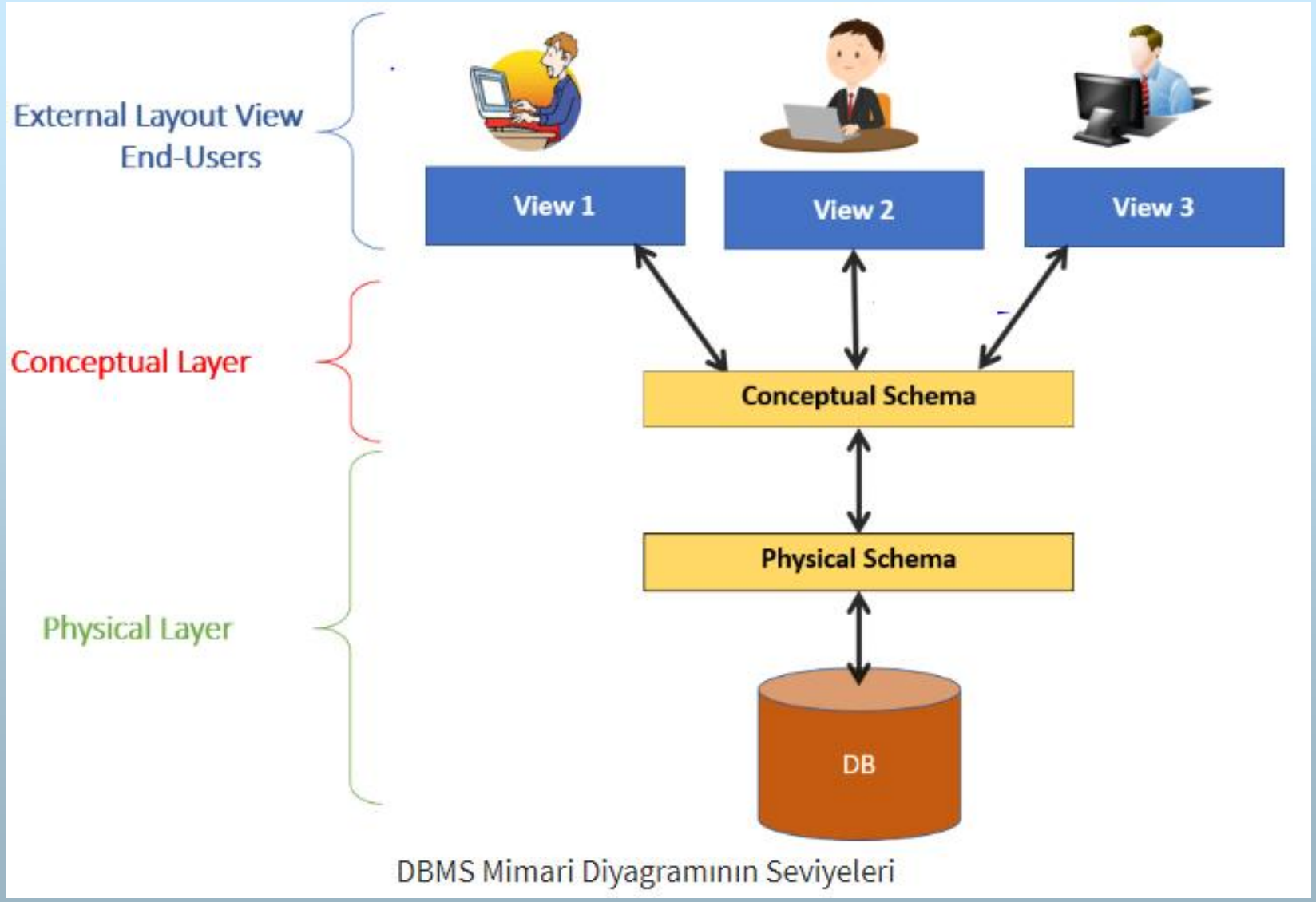
- Görünüm düzeyi: uygulama programları veri türlerinin ayrıntılarını gizler. Görünümler ayrıca güvenlik amacıyla bilgileri (ör. Maaş) gizleyebilir.

Verilerin Görünümü

Bir veritabanı sistemi mimarisi

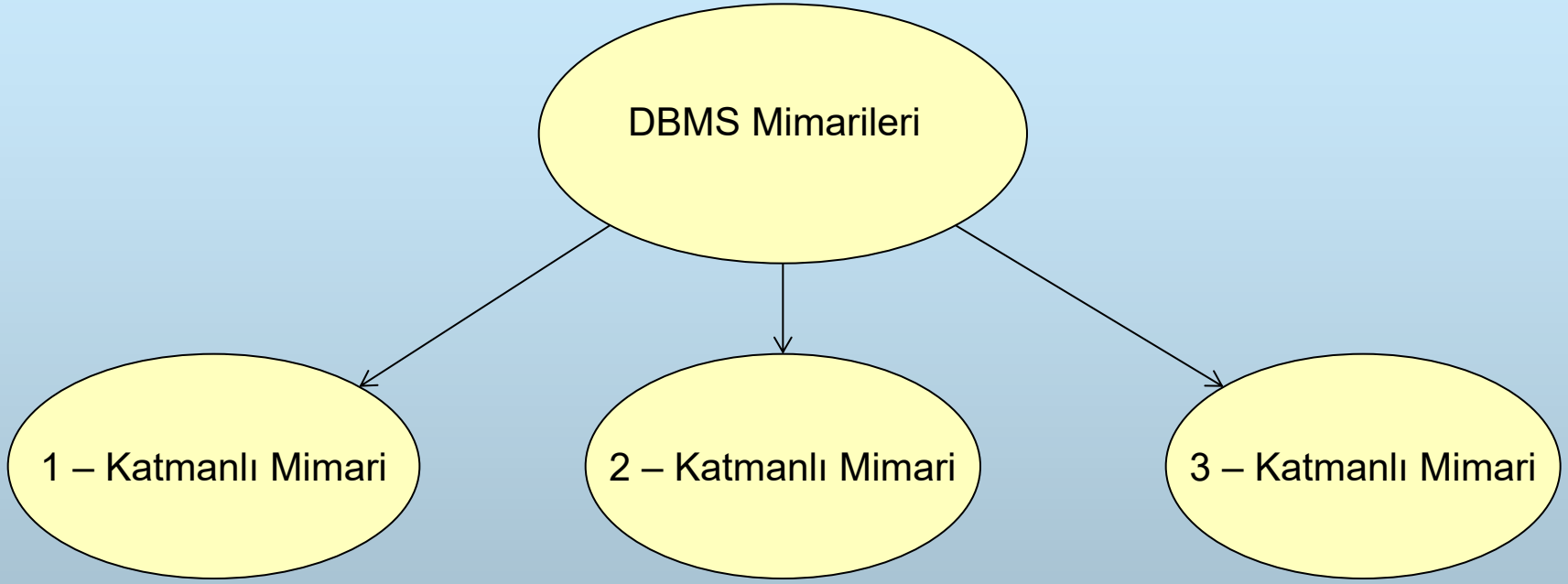


Verilerin Görünümü



DBMS Mimari Diyagramının Seviyeleri

DBMS Mimarisi Türleri



DBMS Mimarisi Türleri

- Veritabanı mimarisi tek katmanlı veya çok katmanlı olarak görülebilir. Ancak mantıksal olarak, veritabanı mimarisi iki türdendir:
  - 2 katmanlı mimari
  - 3 katmanlı mimari .
- 1 Katmanlı Mimari
  - Bu mimaride, veritabanı doğrudan kullanıcı tarafından kullanılabilir. Bu, kullanıcının kullanabileceği anlamına gelir.
  - Burada yapılan herhangi bir değişiklik doğrudan veritabanının kendisinde yapılacaktır. Son kullanıcılar için kullanışlı bir araç sağlamaz.
  - 1-Katmanlı mimari, programcıların hızlı yanıt için doğrudan veritabanı ile iletişim kurabildiği yerel uygulamanın geliştirilmesi için kullanılır.
  - Kullanıcı aryüzü
  - Sunum servisi
  - Uygulama Servisi

DBMS Mimarisi Türleri

- 2 Katmanlı Mimari
  - 2 Katmanlı mimari, temel istemci-sunucu ile aynıdır.
  - İki katmanlı mimaride, istemci tarafındaki uygulamalar, sunucu tarafındaki veritabanı ile doğrudan iletişim kurabilir.
  - Bu etkileşim için API'ler , **ODBC** , **JDBC** gibi kullanılır.
  - Kullanıcı arayüzleri ve uygulama programları, istemci tarafında çalıştırılır.
  - Sunucu tarafı, sorgu işleme ve işlem yönetimi gibi işlevleri sağlamaktan sorumludur.
  - DBMS ile iletişim kurmak için, istemci tarafı uygulaması sunucu tarafı ile bir bağlantı kurar.
  - Doğrudan ve daha hızlı iletişim sağlar.

DBMS Mimarisi Türleri

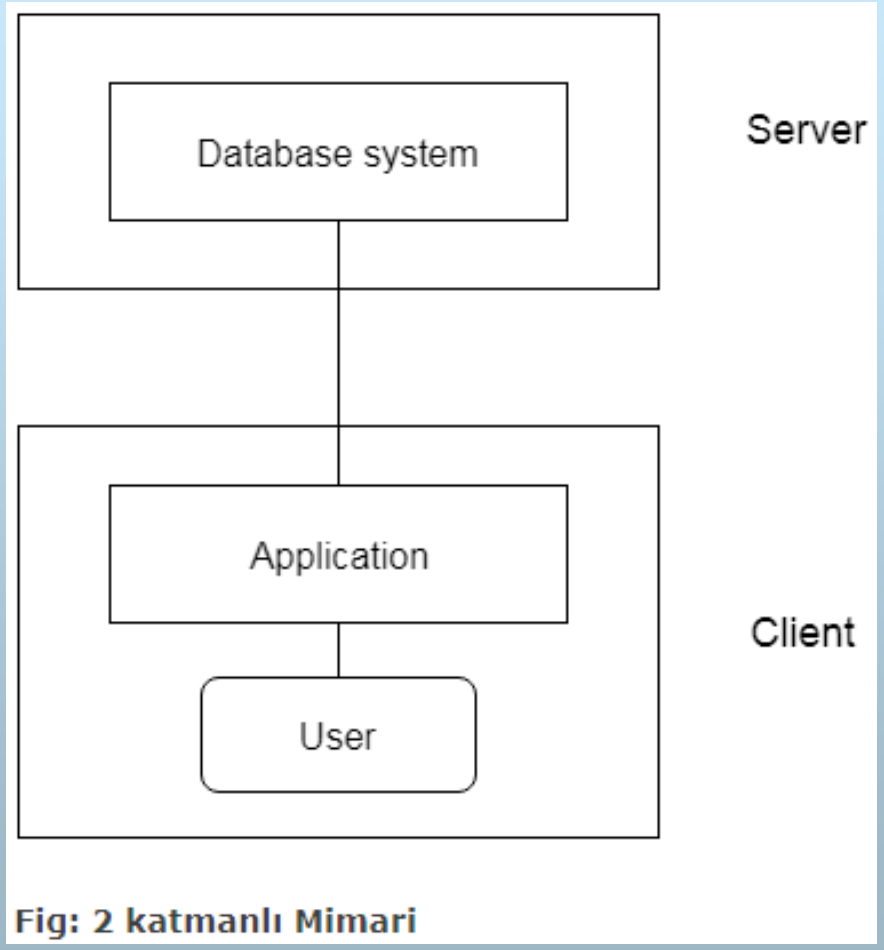
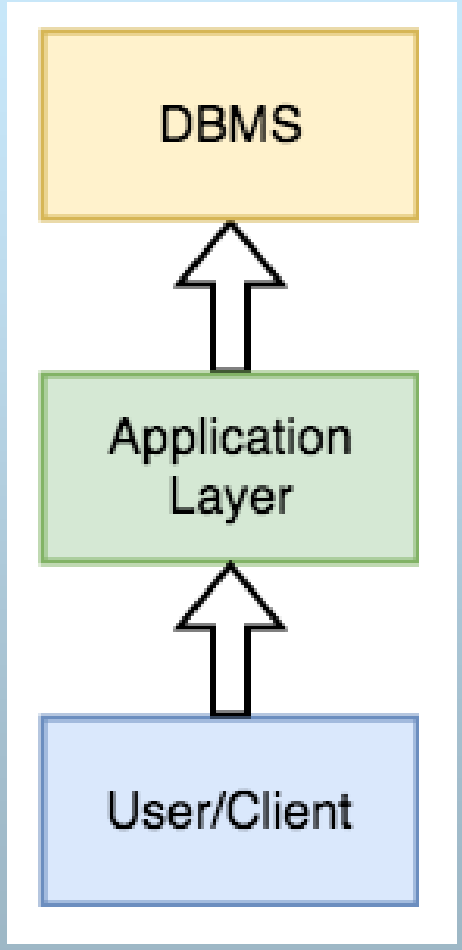


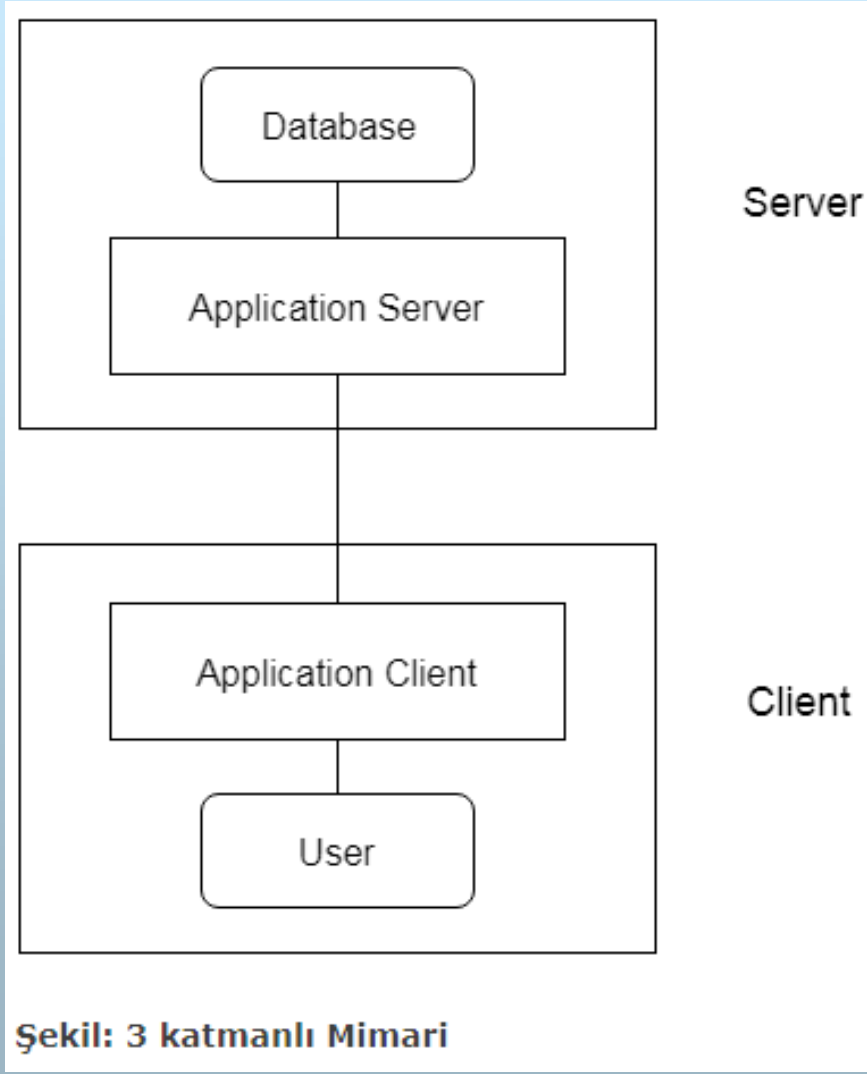
Fig: 2 katmanlı Mimari



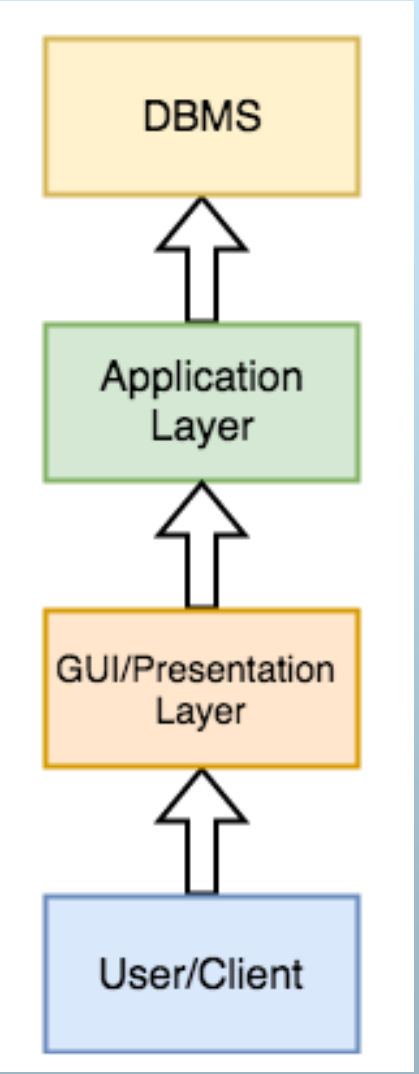
DBMS Mimarisi Türleri

- 3 Katmanlı Mimari
  - 3 Katmanlı mimari, istemci ve sunucu arasında başka bir katman içerir.
  - Bu mimaride, istemci sunucuyla doğrudan iletişim kuramaz.
  - İstemci tarafındaki uygulama, veritabanı sistemiyle daha fazla iletişim kuran bir uygulama sunucusuyla etkileşime girer.
  - Son kullanıcının, veri tabanının uygulama sunucusunun ötesinde varlığı hakkında hiçbir fikri yoktur.
  - Veritabanının ayrıca uygulama dışındaki herhangi bir kullanıcı hakkında hiçbir fikri yoktur.
  - 3 Katmanlı mimari, büyük web uygulaması durumunda kullanılır.

DBMS Mimarisi Türleri



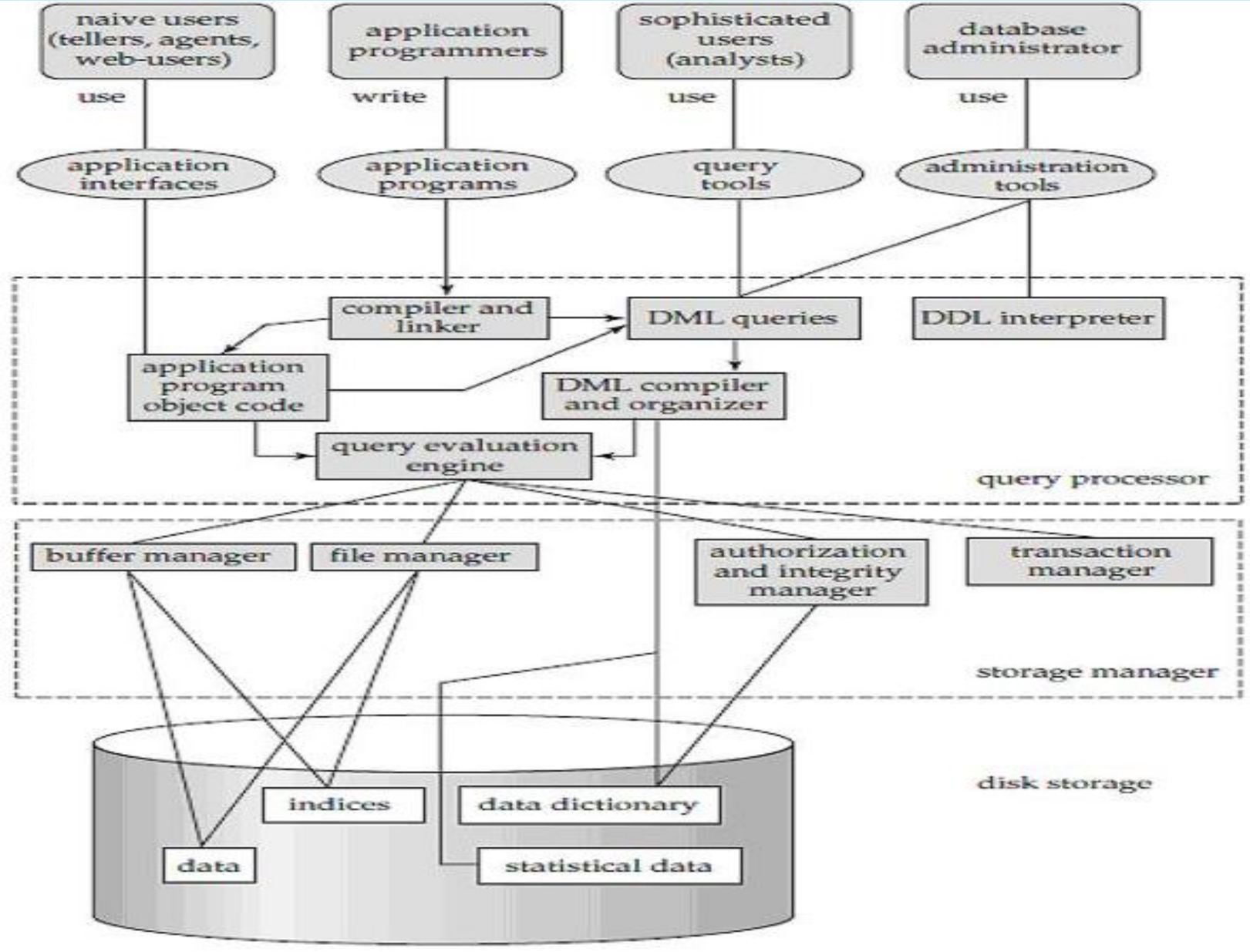
Şekil: 3 katmanlı Mimari



DBMS Mimarisi Türleri

- Database(Data) Katmanı:
  - DB, sorgu işleme diliyle birlikte bulunur
- Uygulama(Middle(Orta)) Katmanı
  - bir kullanıcı için bu, verilerin soyut bir görünümünü sunar
  - kullanıcı ve DB arasında arabulucu görevi görür
- Kullanıcı(Presentation(Sunum)) Katmanı:
  - Son kullanıcı bu katmanda çalışır ve bu katmanın ötesinde DB'nin varlığı hakkında hiçbir şey bilmemektedir

DBMS Mimarisi Türleri



Veritabanı Kullanıcıları ve Yöneticileri

- Yerel Kullanıcı
- Uygulama Programcısı
- Gelişmiş Kullanıcı
- Uzman Kullanıcı
- Online Kullanıcı

Veritabanı Kullanıcıları ve Yöneticileri

- Yerel Kullanıcı
  - DB sisteminin varlığından haberdar olması gerekmeyenler.
  - Veritabanıyla etkileşim kurmak için mevcut uygulamayı kullanan kullanıcılardır
  - Örneğin, mevcut uygulaması olan online kütüphane sistemi, bilet rezervasyon sistemleri, ATM'ler vb
- Uygulama Programcısı
  - Veritabanı ile DML sorguları aracılığıyla etkileşime giren geliştiricilerdir.
  - Bu DML sorguları C, C ++, JAVA , Pascal vb. Uygulama programlarında yazılır .
  - Bu sorgular veritabanı ile iletişim kurmak için nesne koduna dönüştürülür.
    - Örneğin, belirli bir departmanda çalışan çalışanların raporunu oluşturmak için bir C programı yazmak, verileri veritabanından almak için bir sorgu içerecektir.

Veritabanı Kullanıcıları ve Yöneticileri

- Gelişmiş Kullanıcı
  - Veri seçmek / eklemek / silmek / güncellemek için SQL sorguları yazan veritabanı geliştiricileridir.
  - Veritabanını talep etmek için herhangi bir uygulama veya program kullanmazlar.
  - SQL gibi sorgu dili aracılığıyla doğrudan veritabanı ile etkileşime girerler.
  - Bu kategori DBMS ve SQL tasarımcılarını ve geliştiricilerini içerir diyebiliriz.
- Uzman Kullanıcı
  - Bunlar aynı zamanda sofistike kullanıcılardır, ancak özel veritabanı uygulama programları yazarlar.
  - Karmaşık programları ihtiyaca göre geliştiren geliştiricilerdir.
- Online Kullanıcı
  - DB ile online olarak doğrudan iletişim kurabilir.

Veritabanı Kullanıcıları ve Yöneticileri

- Database Yöneticisi
  - DB sistemini uygulamak için bir kişi veya gruptur.
  - DBA, DBMS tarafından izin verilen tüm ayrıcalıklara sahiptir ve yetkileri kullanıcılara atayabilir veya onlardan kaldırabilir.
  - Veritabanı sisteminin tüm faaliyetlerini koordine eder; veritabanı yöneticisi, kuruluşun bilgi kaynaklarını ve ihtiyaçlarını iyi anlar.
  - Veritabanı yöneticisinin görevleri şunları içerir:
    - Şema tanımı
    - Depolama yapısı ve erişim yöntemi tanımı
    - Şema ve fiziksel organizasyon değişikliği
    - Kullanıcıya veritabanına erişim yetkisi verme
    - Bütünlük kısıtlamalarını belirleme
    - Kullanıcılarla irtibat halinde olmak
    - Performansı izleme ve gereksinimlerdeki değişikliklere yanıt verme

İşlem yönetimi

- İşlem, bir veritabanı uygulamasında tek bir mantıksal işlevi gerçekleştiren işlemler koleksiyonudur.
- İşlem yönetimi bileşeni, sistem arızalarına (ör. Güç kesintileri ve işletim sistemi çökmeleri) ve işlem hatalarına rağmen veritabanının tutarlı (doğru) durumda kalmasını sağlar.
- Eşzamanlılık kontrol yöneticisi, veritabanının tutarlılığını sağlamak için eşzamanlı işlemler arasındaki etkileşimi kontrol eder.

Depolama Yönetimi

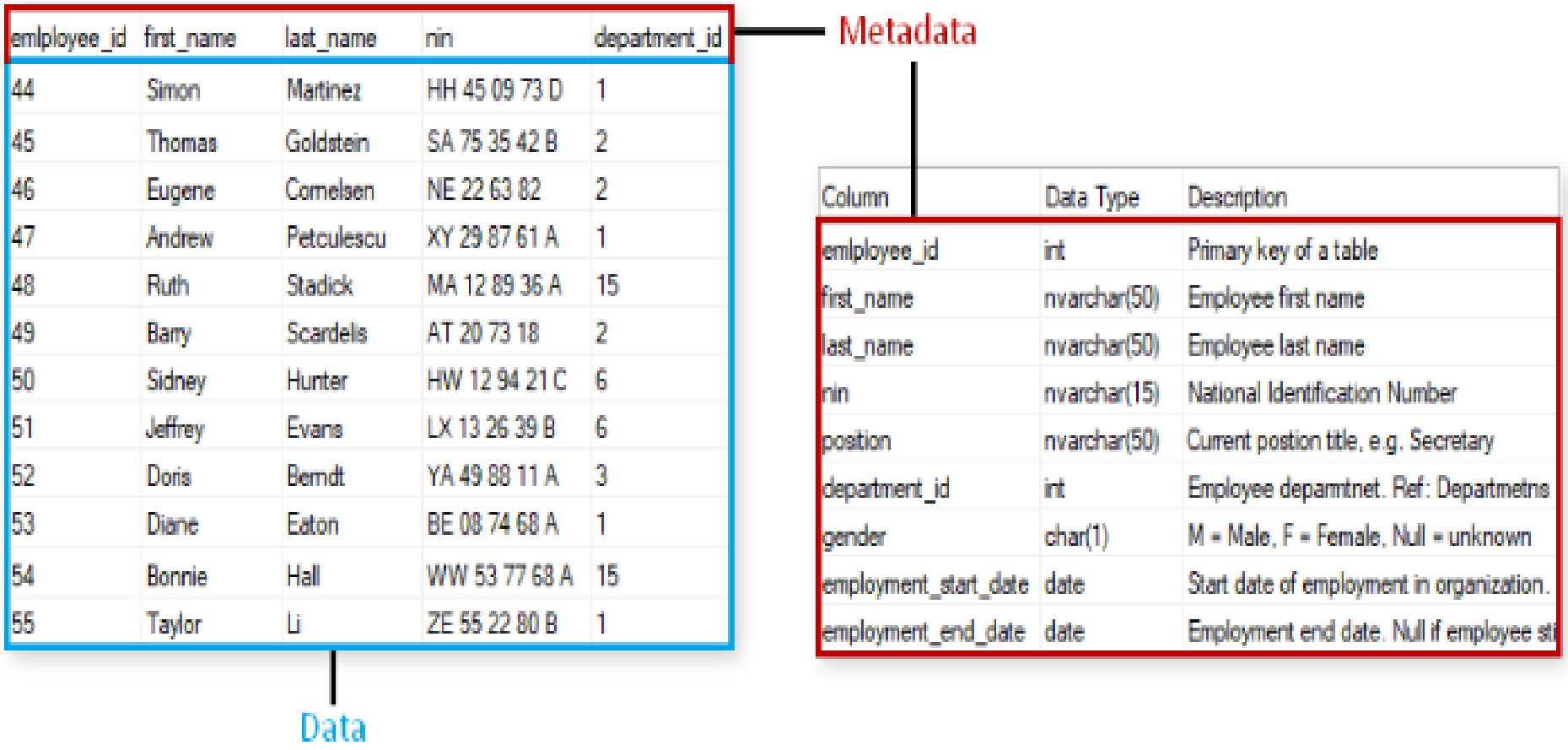
- Depolama yöneticisi, veritabanında depolanan alt düzey veriler ile sisteme gönderilen uygulama programları ve sorgular arasında arayüz sağlayan bir program modülüdür.
- Depolama yöneticisi aşağıdaki görevlerden sorumludur:
  - dosya yöneticisi ile etkileşim
  - Verilerin verimli bir şekilde depolanması, alınması ve güncellenmesi

Veri Bağımsızlığı

- Bir veritabanı sistemi normalde kullanıcıların verilerine ek olarak çok fazla veri içerir.
  - Örneğin, verileri kolayca bulmak ve almak için meta data olarak bilinen verilerle ilgili bilgileri depolar.
- Veritabanında depolandıktan sonra bir dizi meta veriyi değiştirmek veya güncellemek oldukça zordur.
- Ancak bir DBMS genişledikçe, kullanıcıların gereksinimlerini karşılamak için zamanla değişmesi gerekir.
- Tüm veriler bağımlıysa, sistem oldukça karmaşık hale gelir.
- İki tür veri bağımsızlığı vardır:
  - 1- Mantıksal Data Bağımsızlığı
  - 2- Fiziksel Data Bağımsızlığı

Veri Bağımsızlığı(Devam)

- Meta Data



emlployee_id	first_name	last_name	nin	department_id
44	Simon	Martinez	HH 45 09 73 D	1
45	Thomas	Goldstein	SA 75 35 42 B	2
46	Eugene	Cornelsen	NE 22 63 82	2
47	Andrew	Petculescu	XY 29 87 61 A	1
48	Ruth	Stadick	MA 12 89 36 A	15
49	Barry	Scardelis	AT 20 73 18	2
50	Sidney	Hunter	HW 12 94 21 C	6
51	Jeffrey	Evans	LX 13 26 39 B	6
52	Doris	Berndt	YA 49 88 11 A	3
53	Diane	Eaton	BE 08 74 68 A	1
54	Bonnie	Hall	WW 53 77 68 A	15
55	Taylor	Li	ZE 55 22 80 B	1

Column	Data Type	Description
emlployee_id	int	Primary key of a table
first_name	nvarchar(50)	Employee first name
last_name	nvarchar(50)	Employee last name
nin	nvarchar(15)	National Identification Number
postion	nvarchar(50)	Current postion title, e.g. Secretary
department_id	int	Employee deparmtnet. Ref: Departmetns
gender	char(1)	M = Male, F = Female, Null = unknown
employment_start_date	date	Start date of employment in organization.
employment_end_date	date	Employment end date. Null if employee sti

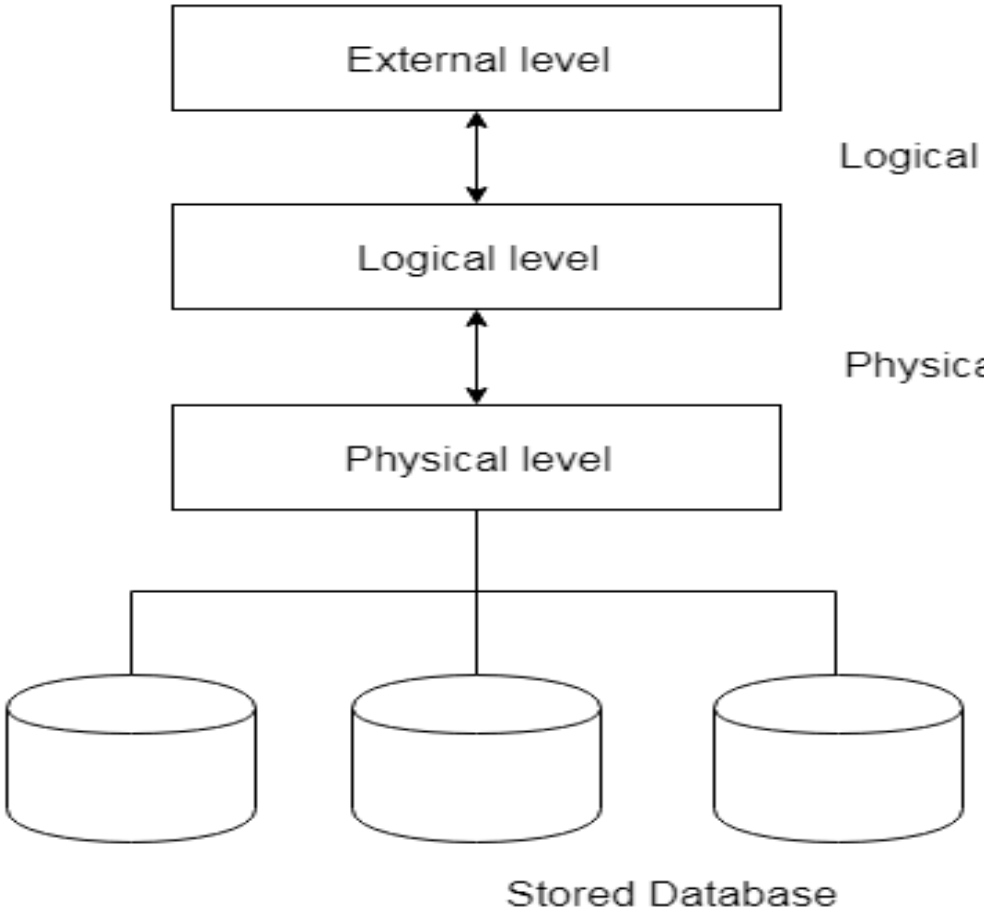
Mantıksal Data Bağımsızlığı

- Mantıksal veriler, veritabanıyla ilgili verilerdir, yani verilerin içinde nasıl yönetildiği hakkında bilgi depolar.
  - Örneğin, veritabanında depolanan bir tablo (ilişki) ve bu ilişkiye uygulanan tüm kısıtlamaları.
- Mantıksal veri bağımsızlığı, kendisini diskte depolanan gerçek verilerden özgürleştiren bir tür mekanizmadır.
- Tablo formatında bazı değişiklikler yaparsak, diskte bulunan verileri değiştirmemelidir.
- Mantıksal Veri Bağımsızlığı altındaki değişiklik örnekleri
  - Mevcut uygulama programlarını yeniden yazmadan yeni bir öznitelik, varlık veya ilişki Ekleme / Değiştirme / Silme mümkündür
  - İki kaydı tek bir kayıtta birleştirme
  - Mevcut bir kaydı iki veya daha fazla kayda bölme

Fiziksel Data Bağımsızlığı

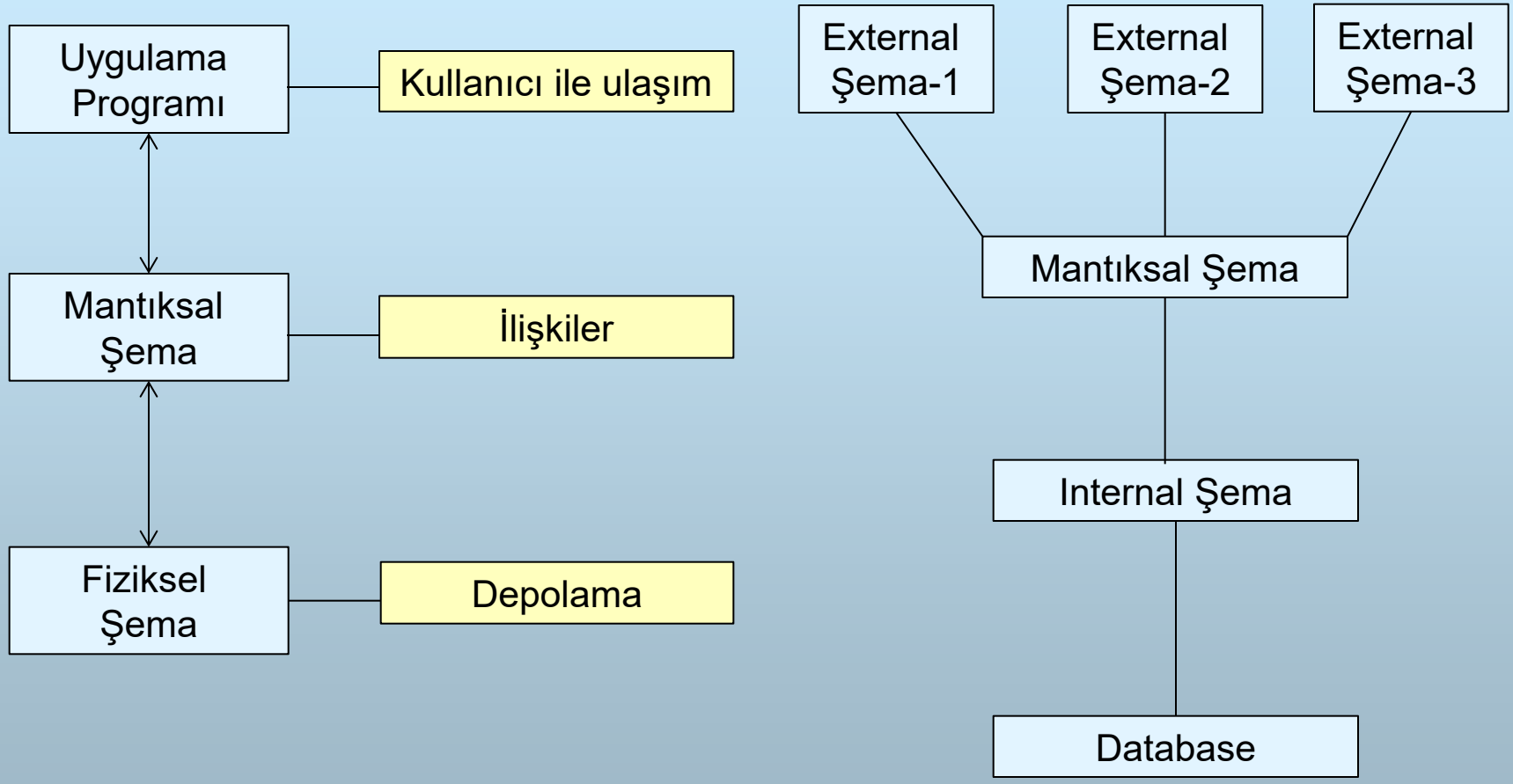
- Tüm şemalar mantıksaldır ve gerçek veriler diskte bit formatında saklanır.
- Fiziksel veri bağımsızlığı, şemayı veya mantıksal verileri etkilemeden fiziksel verileri değiştirme gücüdür.
  - Örneğin, depolama sisteminin kendisini değiştirmek veya yükseltmek istememiz durumunda - sabit diskleri SSD ile değiştirmek istediğimizi varsayalım - mantıksal veriler veya şemalar üzerinde herhangi bir etkisi olmamalıdır.
- Fiziksel Veri Bağımsızlığı kapsamındaki değişiklik örnekleri
  - Sabit Disk veya Manyetik Bantlar gibi yeni bir depolama cihazı kullanma
  - Veritabanında dosya düzenleme tekniğini değiştirme
  - Erişim yöntemini değiştirme.
  - Dizinleri değiştirme.
  - Veritabanının Konumunun C sürücüsünden D Sürücüsüne değiştirilmesi

Data Bağımsızlığı



Şekil: Veri Bağımsızlığı

Data Bağımsızlığı



Örnekler ve Şemalar

- Programlama dillerindeki türlere ve değişkenlere benzer.
- **Şema** – veritabanının mantıksal yapısı
  - Örneğin, veritabanı bir dizi müşteri ve hesap ile aralarındaki ilişkiyle ilgili bilgilerden oluşur
  - Bir programdaki bir değişkenin bilgisinin yazılmasına benzer
  - **Fiziksel şema**: fiziksel düzeyde veritabanı tasarımı
  - **Mantıksal şema**:mantıksal düzeyde veritabanı tasarımı
- **Örnek** – veri tabanının belirli bir zamandaki gerçek içeriği
  - Bir değişkenin değerine benzer
- **Fiziksel Veri Bağımsızlığı**– mantıksal şemayı değiştirmeden fiziksel şemayı değiştirme yeteneği
  - Uygulamalar mantıksal şemaya bağlıdır
  - Genel olarak, çeşitli seviyeler ve bileşenler arasındaki arayüzler, bazı kısımlardaki değişikliklerin diğerlerini ciddi şekilde etkilememesi için iyi tanımlanmalıdır.

Data Modelleri

- Açıklamak için bir araç koleksiyonu
  - data
  - data ilişkileri
  - veri anlambilim
  - veri kısıtlamaları
- Varlık-İlişki modeli(Entity-Relationship model)
- İlişkisel Model (Relational model)
- Diğer modeller:
  - object-oriented model
  - semi-structured data model
  - Daha eski modeller: network model ve hierarchical model

Data Modelleri

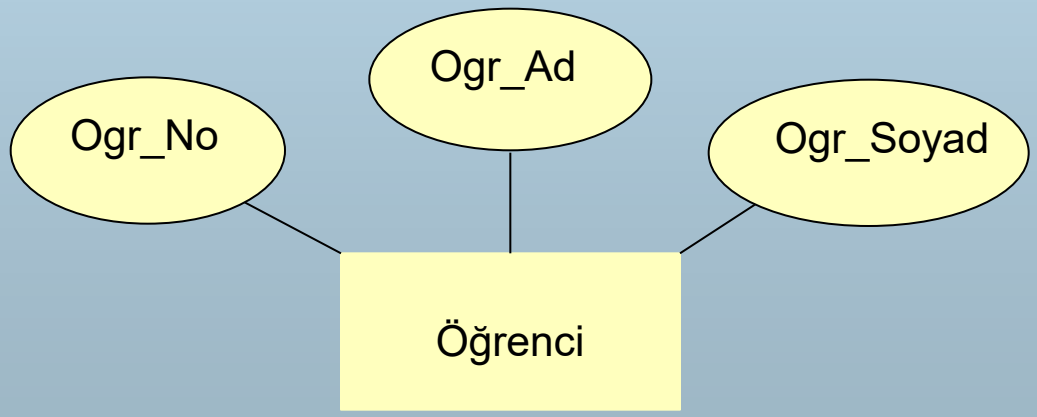
- Açıklamak için bir araç koleksiyonu
  - data
  - data ilişkileri
  - veri anlambilim
  - veri kısıtlamaları
- Varlık-İlişki modeli(Entity-Relationship model)
- İlişkisel Model (Relational model)
- Diğer modeller:
  - object-oriented model
  - semi-structured data model
  - Daha eski modeller: network model ve hierarchical model

Entity-Relationship Model

- ER modeli, bir veritabanının kavramsal görünümünü tanımlar.
- Gerçek dünyadaki varlıklar ve aralarındaki ilişkiler etrafında çalışır.
- Görünüm düzeyinde, ER modeli, veritabanları tasarlamak için iyi bir seçenek olarak kabul edilir.
- Varlık(Entity):
  - Bir varlık, canlı veya cansız, kolayca tanımlanabilen gerçek dünya nesnesi olabilir.
    - Örneğin, bir okul veritabanında öğrenciler, öğretmenler, sınıflar ve sunulan dersler varlıklar olarak kabul edilebilir.
    - Tüm bu varlıklar, onlara kimliklerini veren bazı niteliklere veya özelliklere sahiptir.
  - Bir varlık grubu(**Entity Set**), benzer türde varlıkların bir koleksiyonudur
  - Bir varlık grubu, benzer değerleri paylaşan özniteliğe sahip varlıklar içerebilir.
    - Örneğin, bir Öğrenci kümesi bir okulun tüm öğrencilerini içerebilir.

Entity-Relationship Model(Devam)

- Öznitellikler(Attributes)
  - Varlıklar, **öznitelik** adı verilen **özellikleri** aracılığıyla temsil edilir .
  - Tüm özniteliklerin değerleri vardır.
    - Örneğin, bir öğrenci varlığının nitelik olarak adı, sınıfı ve yaşı olabilir.
  - Özniteliklere atanabilecek bir alan(domain) veya değerler aralığı vardır.
    - Örneğin, bir öğrencinin adı sayısal bir değer olamaz. Alfabetik olmalıdır. Bir öğrencinin yaşı negatif olamaz vb.

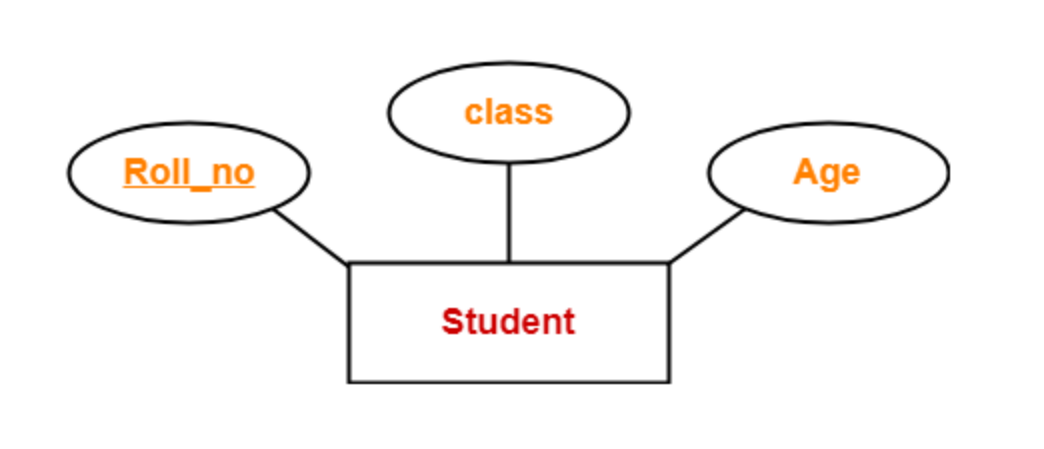


Entity-Relationship Model (Devam)

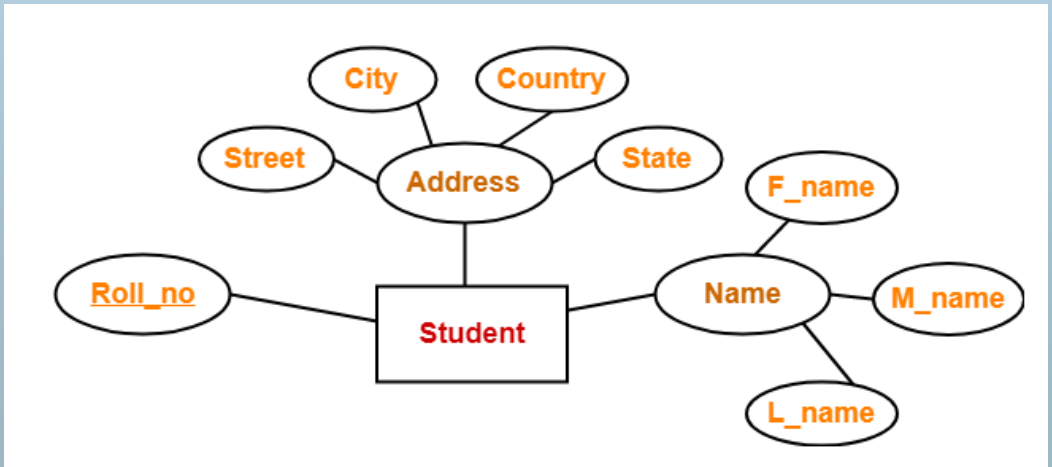
- Öznitelik Türler(Type of Attributes)
  - Basit öznitelik
    - Basit öznitelikler, daha fazla bölünemeyen atomik değerlerdir.
      - Örneğin, bir öğrencinin telefon numarası, 10 basamaklı bir atomik değerdir.
  - Bileşik öznitelik
    - Bileşik öznitelikler, birden fazla basit öznitelikten oluşur.
      - Örneğin, bir öğrencinin tam adı ve soyadı olabilir.
  - Türetilmiş öznitelik
    - Türetilmiş öznitelikler, fiziksel veritabanında bulunmayan özniteliklerdir, ancak değerleri veritabanında bulunan diğer özniteliklerden türetilir.
  - Tek değerli öznitelik
    - Tek değerli öznitelikler tek bir değer içerir.
  - Çok değerli öznitelik
    - Çok değerli öznitelikler birden fazla değer içerebilir.

Entity-Relationship Model (Devam)

- Basit öznitelik

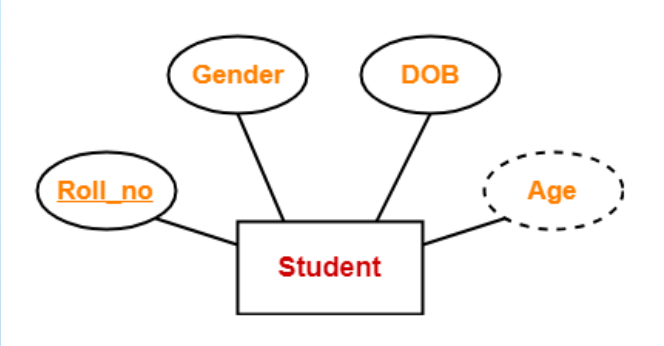


- Bileşik öznitelik

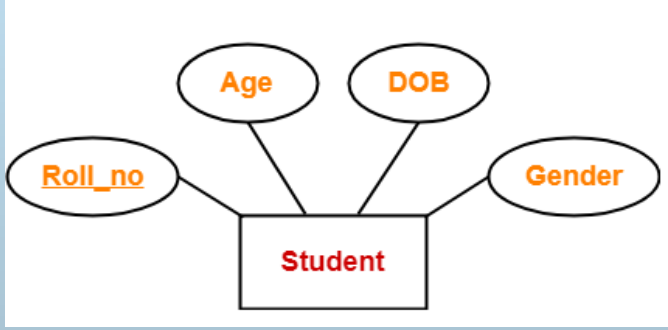


Entity-Relationship Model (Devam)

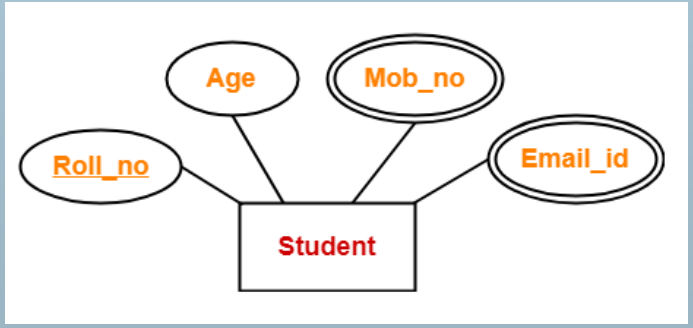
- Türetilmiş öznitelik



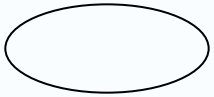
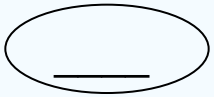
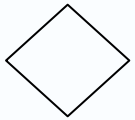
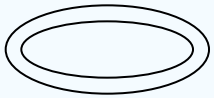
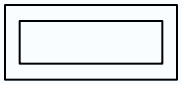
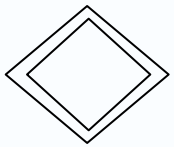
- Tek değerli öznitelik



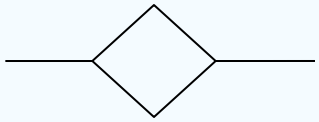
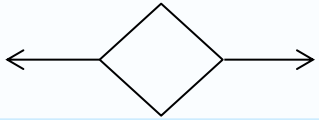
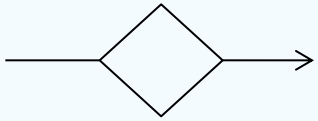
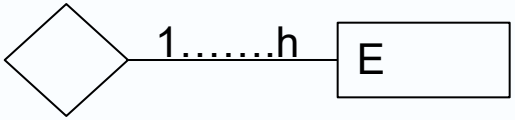
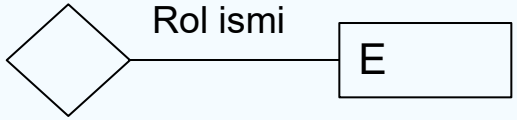
- Çok değerli öznitelik



ER Diyagramının Bileşenleri

Sembol	Açıklama
	Varlık Kümesi Seti
	Nitelik
	Anahtar Nitelik
	İlişki
	Çok değerli nitelik
	Zayıf Varlık Kümesi
	Türetilmiş nitelik
	Zayıf İlişki

ER Diyagramının Bileşenleri(Devam)

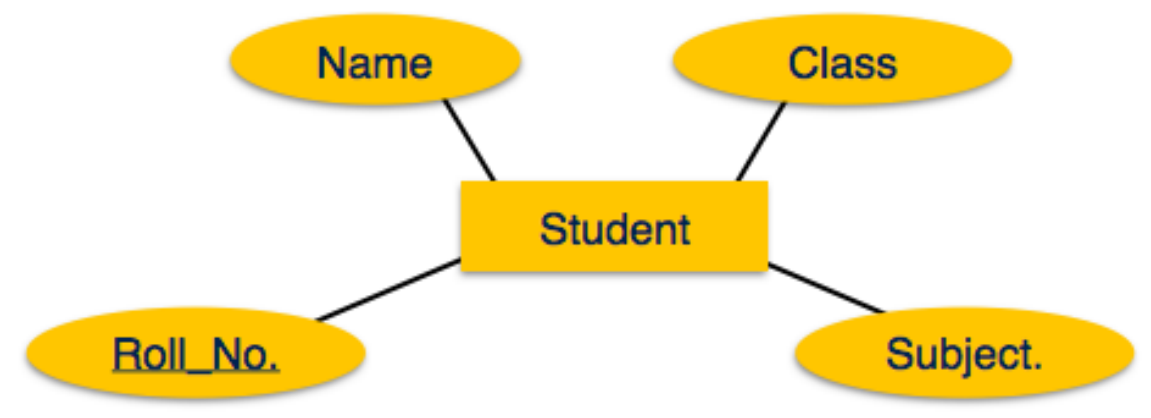
Sembol	Açıklama
	Çoka – Çok İlişki
	Bire – Bir İlişki
	Çoktan – Bire İlişki
	Nicelik limitleri
	Rol göstergesi

İlişkisel Modelden ER Modeli

- ER Modeli, diyagramlar halinde kavramsallaştırıldığında, anlaşılması daha kolay olan varlık-ilişkisine iyi bir genel bakış sağlar.
- ER diyagramları ilişkisel şema ile eşleştirilebilir, yani ER diyagramı kullanılarak ilişkisel şema oluşturmak mümkündür.
- Tüm ER kısıtlamalarını ilişkisel modele aktaramayız, ancak yaklaşık bir şema oluşturulabilir.
- ER Diyagramlarını İlişkisel Şemaya dönüştürmek için kullanılabilen birkaç işlem ve algoritma vardır.
- ER diyagramları esas olarak şunlardan oluşur:
  - Varlık ve öznitelikleri
  - Varlıklar arasındaki ilişki olan bağlantı.

İlişkisel Modelden ER Modeli

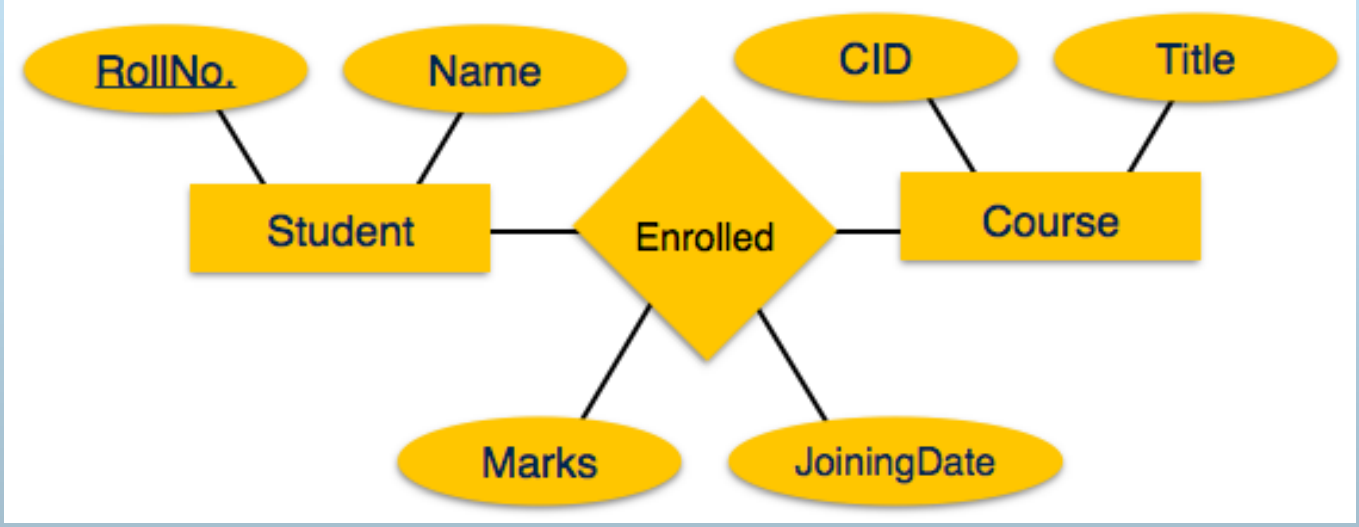
- Varlık Haritalama
  - Bir varlık, bazı niteliklere sahip gerçek dünya nesnesidir.



  - Haritalama Süreci (Algoritma)
    - Her varlık için tablo oluşturulur.
    - Varlığın öznitelikleri, ilgili veri türleriyle birlikte tablo alanları haline gelmelidir.
    - Birincil anahtar belirtilir

İlişkisel Modelden ER Modeli

- İlişki Haritalama
  - Bir ilişki, varlıklar arasındaki bir ilişkidir.

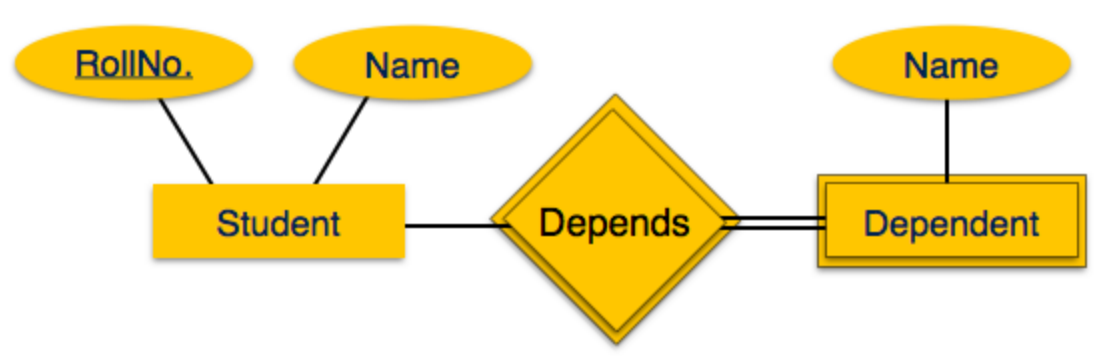


İlişkisel Modelden ER Modeli

- İlişki Haritalama
  - Haritalama Süreci
    - Bir ilişki için tablo oluşturulması.
    - Tüm katılımcı Varlıkların birincil anahtarlarını ilgili veri türleriyle birlikte tablo alanları olarak eklenmesi.
    - İlişkinin herhangi bir özniteliği varsa, her özniteliğin tablo alanı olarak eklenmesi.
    - Katılımcı varlıkların tüm birincil anahtarlarını oluşturan bir birincil anahtarın belirtilmesi.
    - Tüm yabancı anahtar kısıtlamalarının belirtilmesi.

İlişkisel Modelden ER Modeli

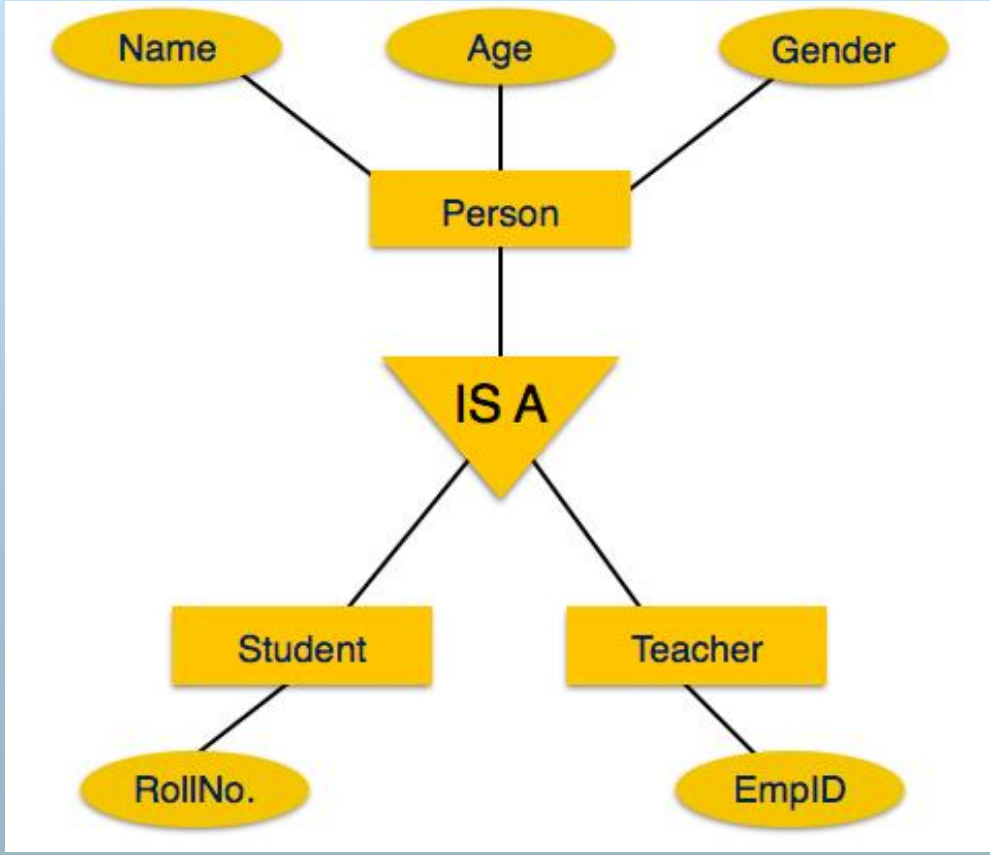
- Zayıf Varlık Kümelerini Haritalama
  - Zayıf bir iletişim kümesi, kendisiyle herhangi bir birincil anahtara sahip olmayan bir iletişim kümesidir.



  - Haritalama Süreci
    - Zayıf varlık kümesi için tablonun oluşturulması.
    - Tüm özelliklerini tabloya alan olarak eklenmesi.
    - Varlık kümesini tanımlayan birincil anahtarın eklenmesi.
    - Tüm yabancı anahtar kısıtlamalarının belirtilmesi.

İlişkisel Modelden ER Modeli

- Hiyerarşik Varlıkları Haritalama
  - ER özelleşme veya genellemesi, hiyerarşik varlık kümeleri biçiminde gelir.

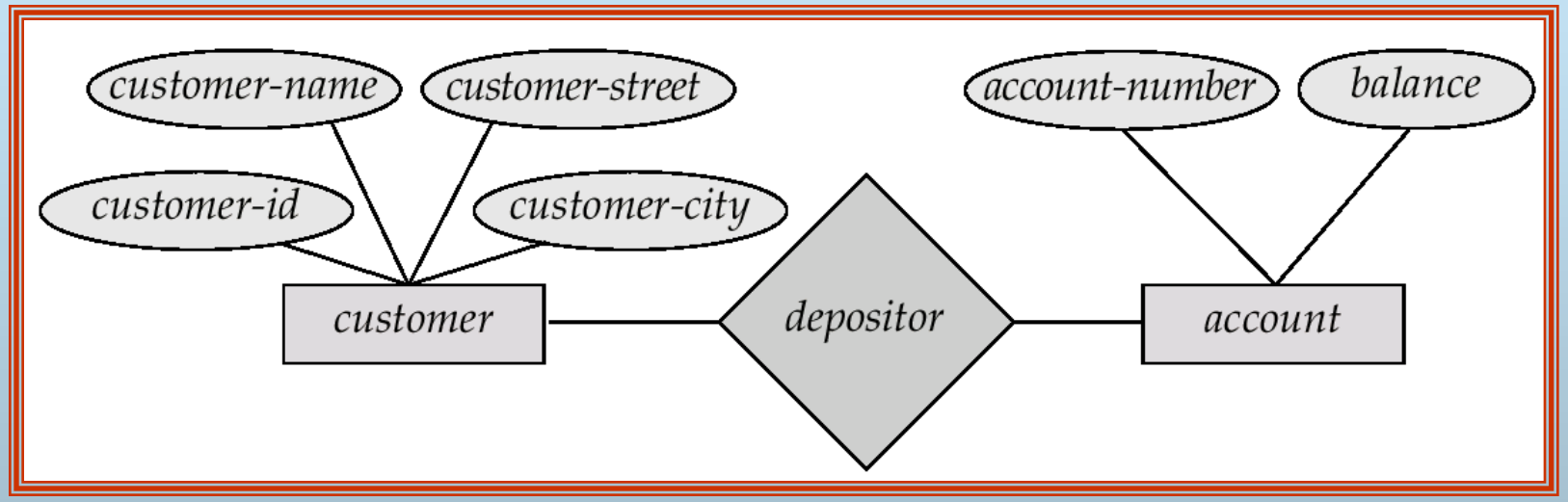


İlişkisel Modelden ER Modeli

- Hiyerarşik Varlıkları Haritalama(Devam)
  - Haritalama Süreci
    - Tüm üst düzey varlıklar için tablolar oluşturulur.
    - Daha düşük seviyeli varlıklar için tablolar oluşturulur.
    - Daha düşük seviyeli varlıklar tablosuna daha yüksek seviyeli varlıkların birincil anahtarları eklenir.
    - Alt düzey tablolarda, alt düzey varlıkların diğer tüm özniteliklerinin eklenmesi.
    - Üst düzey tablonun birincil anahtarını ve daha düşük düzeyli tablonun birincil anahtarının belirtilmesi.
    - Yabancı anahtar kısıtlamalarının belirtilmesi.

Entity-Relationship Model(Devam)

Varlık-ilişki modelinde şema örneği



İlişkisel Model(Relational Mode)

- İlişkisel veri modeli, veri depolama ve işleme için dünya çapında yaygın olarak kullanılan birincil veri modelidir.
- Bu model basittir ve verileri depolama verimliliği ile işlemek için gereken tüm özelliklere ve yeteneklere sahiptir.
- Kavramlar
  - **Tablolar(Tables)**
    - İlişkisel veri modelinde ilişkiler Tablolar formatında kaydedilir.
    - Bu format, varlıklar arasındaki ilişkiyi saklar.
    - Bir tabloda, satırların kayıtları ve sütunların öznitelikleri temsil ettiği satırlar ve sütunlar vardır.
  - **Tuple**
    - Bu ilişki için tek bir kayıt içeren bir tablonun tek satırına demet adı verilir.

İlişkisel Model(Relational Mode)

Informal Terimler	Formal Terimler
Tablo(Table)	Relation(İlişki)
Kolon başlık / alan (Column header/field)	Öznitelikler(Attributes)
Olası tüm sütun değerleri (All possible column valıes)	Domain
Satır(Row)	Tuple
Tablo tanımı(Table definition)	İlişki şeması (Schema of Relation)

- İlişki derecesi nasıl bulunur: Bir ilişkideki niteliklerin sayısına derece denir.
- İlişkinin önemi(niceliği) nasıl bulunur?: Bir ilişkideki tupellerin sayısına nicelik(önem)denir.

İlişkisel Model(Relational Mode)

- İlişki Özellikleri(Properties of Relation)
  - İlişkinin, ilişki şemasındaki tüm adlardan farklı bir adı vardır.
  - İlişkinin her hücresi tam olarak bir değer içerir.
  - Her özniteliğin farklı adları vardır.
  - Her demet(tuple) farklıdır, yinelenen demetler yoktur.
  - Öz nitelik sırasının bir önemi yoktur
  - Demet(tuple) sırasının önemi yoktur..

İlişkisel Model(Relational Mode)

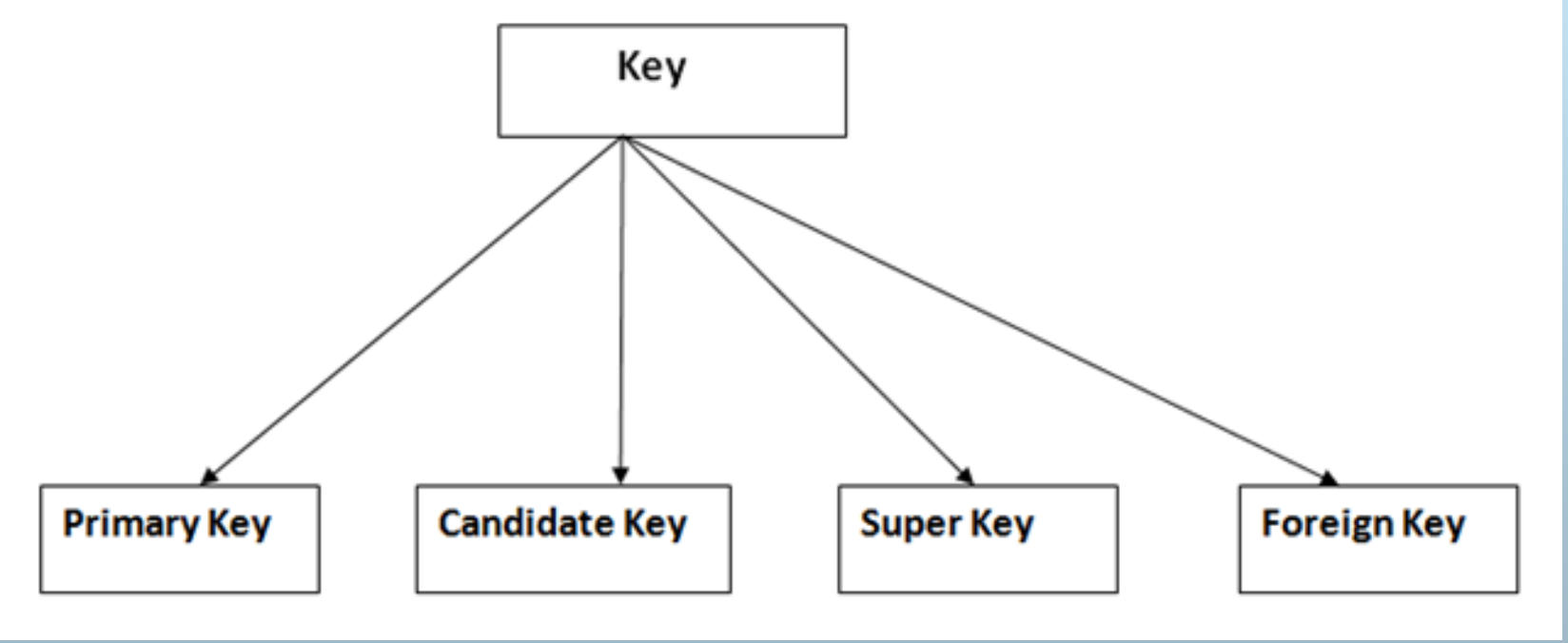
- Kavramlar(devam)
  - **İlişki örneği(Relation instance)**
    - İlişkisel veritabanı sistemindeki sonlu bir grup kümesi, ilişki örneğini temsil eder.
    - İlişki örneklerinin yinelenen demetleri yoktur.
  - **İlişki şeması(Relation schema)**
    - Bir ilişki şeması, ilişki adını (tablo adı), öznitelikleri ve adlarını açıklar.
  - **İlişki anahtarı(Relation key)**
    - Her satırın, ilişkideki (tablodaki) satırı benzersiz bir şekilde tanımlayabilen, ilişki anahtarı olarak bilinen bir veya daha fazla özniteliği vardır.
  - **Öznitelik etki alanı(Attribute domain)**
    - Her özniteliğin, öznitelik etki alanı olarak bilinen önceden tanımlanmış bir değer kapsamı vardır.

İlişkisel Model(Relational Mode)

- Kısıtlamalar
  - Her ilişkinin, geçerli bir ilişki olması için tutması gereken bazı koşulları vardır. Bu koşullara **İlişkisel Bütünlük Kısıtlamaları** denir .
  - Üç ana bütünlük kısıtlaması vardır
    - Anahtar kısıtlamalar
      - bir anahtar öznitelik ile bir ilişkide, iki demet anahtar öznitelikleri için aynı değerlere sahip olamaz.
      - bir anahtar özelliği NULL değerlere sahip olamaz.
      - Anahtar kısıtlamalara **Varlık** Kısıtlamaları da denir.
    - Etki alanı kısıtlamaları
      - Niteliklerin gerçek dünya senaryosunda belirli değerleri vardır. Örneğin, yaş yalnızca pozitif bir tam sayı olabilir.
    - Bilgi tutarlılığı kısıtlamaları
      - Bilgi tutarlılığı kısıtlamaları, Yabancı Anahtarlar kavramı üzerinde çalışır. Yabancı anahtar, başka bir ilişkide başvurulabilen bir ilişkinin temel bir özelliğidir.

DBMS'de Anahtar Kelimeler

- Anahtar Türleri



İlişkisel Model(Relational Mode)

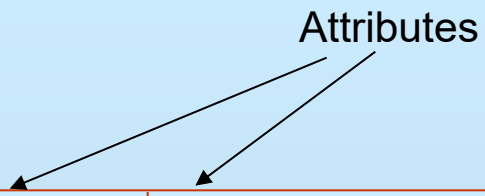
- Anahtar Türleri
  - Birincil anahtar
    - Bir varlığın bir ve yalnızca bir örneğini benzersiz olarak tanımlamak için kullanılan ilk anahtardır.
  - Aday anahtarı
    - Bir aday anahtar, bir demeti benzersiz bir şekilde tanımlayabilen bir öznitelik veya bir öznitelik kümesidir.
    - Birincil anahtar dışında kalan öznitelikler bir aday anahtar olarak kabul edilir.
    - Aday anahtarlar, birincil anahtar kadar güçlüdür.
  - Süper Anahtar
    - Süper anahtar, bir demeti benzersiz şekilde tanımlayabilen bir öznitelik kümesidir. Süper anahtar, aday anahtarın üst kümesidir
  - Yabancı anahtar
    - Yabancı anahtarlar, başka bir tablonun birincil anahtarına işaret etmek için kullanılan tablonun sütunudur.

İlişkisel Model(Relational Mode)

- Anahtar Türleri(Devam)
  - İkincil Anahtar / Alternatif(Secondary Key)
  - Bileşik Anahtar(Composite Key)
  - Benzersiz Anahtar(Unique Key)
    - NULL değeri alır

İlişkisel Model(Relational Model)

- İlişkisel modeldeki tablo verisi örneği



Customer-id	*customer-name*	*customer-street*	*customer-city*	*account- number*
192-83-7465	Johnson	Alma	Palo Alto	A-101
019-28-3746	Smith	North	Rye	A-215
192-83-7465	Johnson	Alma	Palo Alto	A-201
321-12-3123	Jones	Main	Harrison	A-217
019-28-3746	Smith	North	Rye	A-201

Basit Bir İlişkisel Veritabanı

customer-id	*customer-name*	*customer-street*	*customer-city*
192-83-7465	Johnson	12 Alma St.	Palo Alto
019-28-3746	Smith	4 North St.	Rye
677-89-9011	Hayes	3 Main St.	Harrison
182-73-6091	Turner	123 Putnam Ave.	Stamford
321-12-3123	Jones	100 Main St.	Harrison
336-66-9999	Lindsay	175 Park Ave.	Pittsfield
019-28-3746	Smith	72 North St.	Rye

(a) The *customer* table

account-number	*balance*
A-101	500
A-215	700
A-102	400
A-305	350
A-201	900
A-217	750
A-222	700

(b) The *account* table

customer-id	*account-number*
192-83-7465	A-101
192-83-7465	A-201
019-28-3746	A-215
677-89-9011	A-102
182-73-6091	A-305
321-12-3123	A-217
336-66-9999	A-222
019-28-3746	A-201

(c) The *depositor* table

Veri Tanımlama Dili(DDL)

- Veri Tanımlama Dili, veritabanı yapısını veya şemasını tanımlamanıza yardımcı olur
  - Ötneğin

    create table *account* (
    account-number **char**(10),
    balance **integer**)
- DDL derleyici, bir veri sözlüğünde depolanan bir dizi tablo oluşturur
- Veri sözlüğü meta verileri içerir (yani verilerle ilgili veriler)
  - database şema
  - Veri depolama ve tanımlama dili
    - veritabanı sistemi tarafından kullanılan depolama yapısı ve erişim yöntemlerinin belirtildiği dil
    - Genellikle veri tanımlama dilinin bir uzantısı

Veri Tanımlama Dili(DDL Devam)

- İşte DDL kapsamında gelen bazı komutlar:
  - CREATE
  - ALTER
  - DROP
  - TRUNCATE

Veri İşleme Dili(DML)

- Uygun veri modeli tarafından düzenlenen verilere erişme ve bunları değiştirme dili
  - DML, sorgu dili olarak da bilinir
- İki sınıf dil
  - Prosedürel - kullanıcı hangi verilerin gerekli olduğunu ve bu verilerin nasıl alınacağını belirtir
  - Nonprocedural – user specifies what data is required without specifying how to get those data
  - Nonprocedural - kullanıcı, bu verilerin nasıl alınacağını belirtmeden hangi verilerin gerekli olduğunu belirtir
- SQL, en yaygın kullanılan sorgu dilidir
- DML altında gelen bazı komutlar:
  - INSERT,
  - UPDATE
  - DELETE

Structured Query Language(SQL)

- SQL: yaygın olarak kullanılan prosedürel olmayan dil
  - Örneğin. 192-83-7465 müşteri kimliğine sahip müşterinin adını bulun

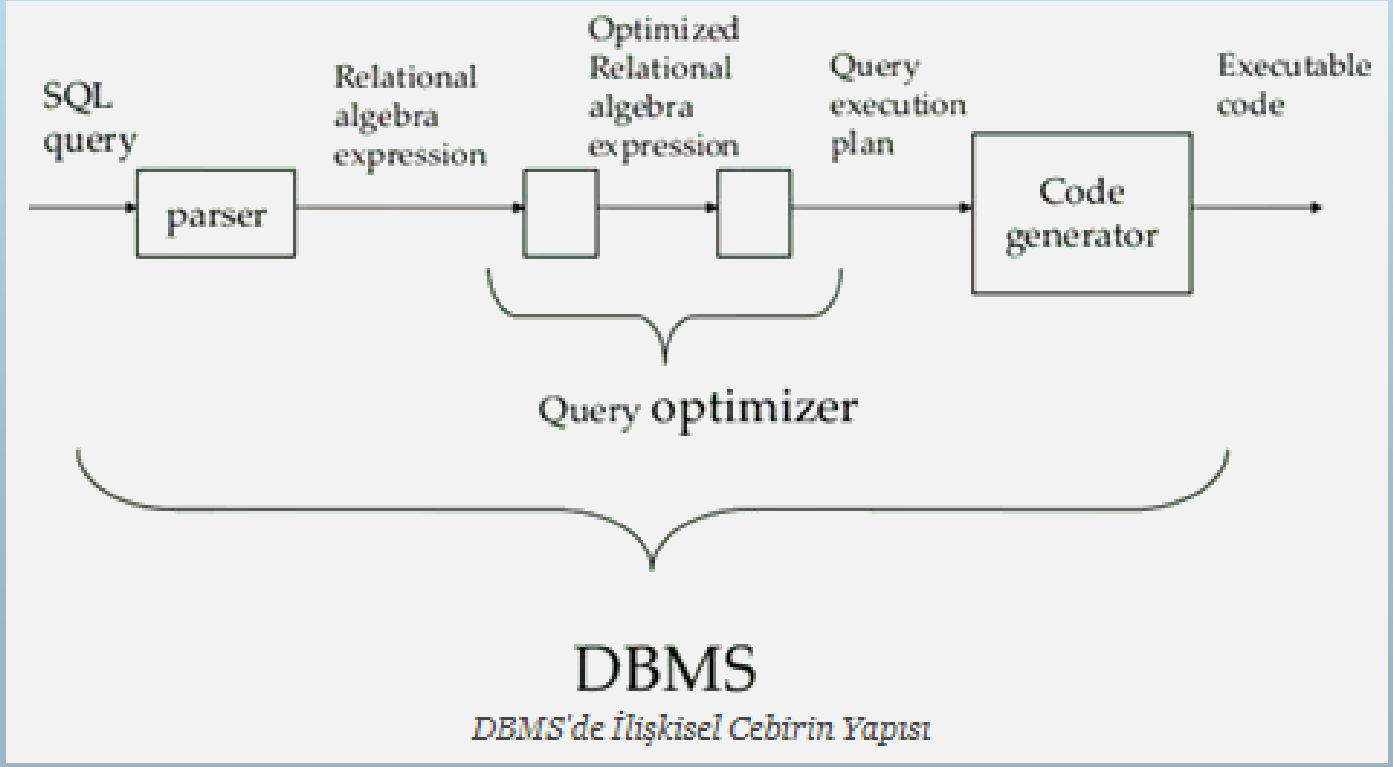
    select *customer.customer-name*
    from *customer*
    where *customer.customer-id* = ‘192-83-7465’
  - Örneğin. 192-83-7465 müşteri kimliğine sahip müşterinin tuttuğu tüm hesapların bakiyelerini bulun

    select *account.balance*
    from *depositor*, *account*
    where *depositor.customer-id* = ‘192-83-7465’ **and**
    depositor.account-number = account.account-number
- Uygulama programları genellikle veri tabanlarına aşağıdakilerden biri aracılığıyla erişir:
  - Gömülü SQL'e izin veren dil uzantıları
  - SQL sorgularının bir veritabanına gönderilmesine izin veren uygulama programı arayüzü (örneğin ODBC / JDBC)

İLİŞKİSEL CEBİR
(RELATIONAL ALGEBRA)

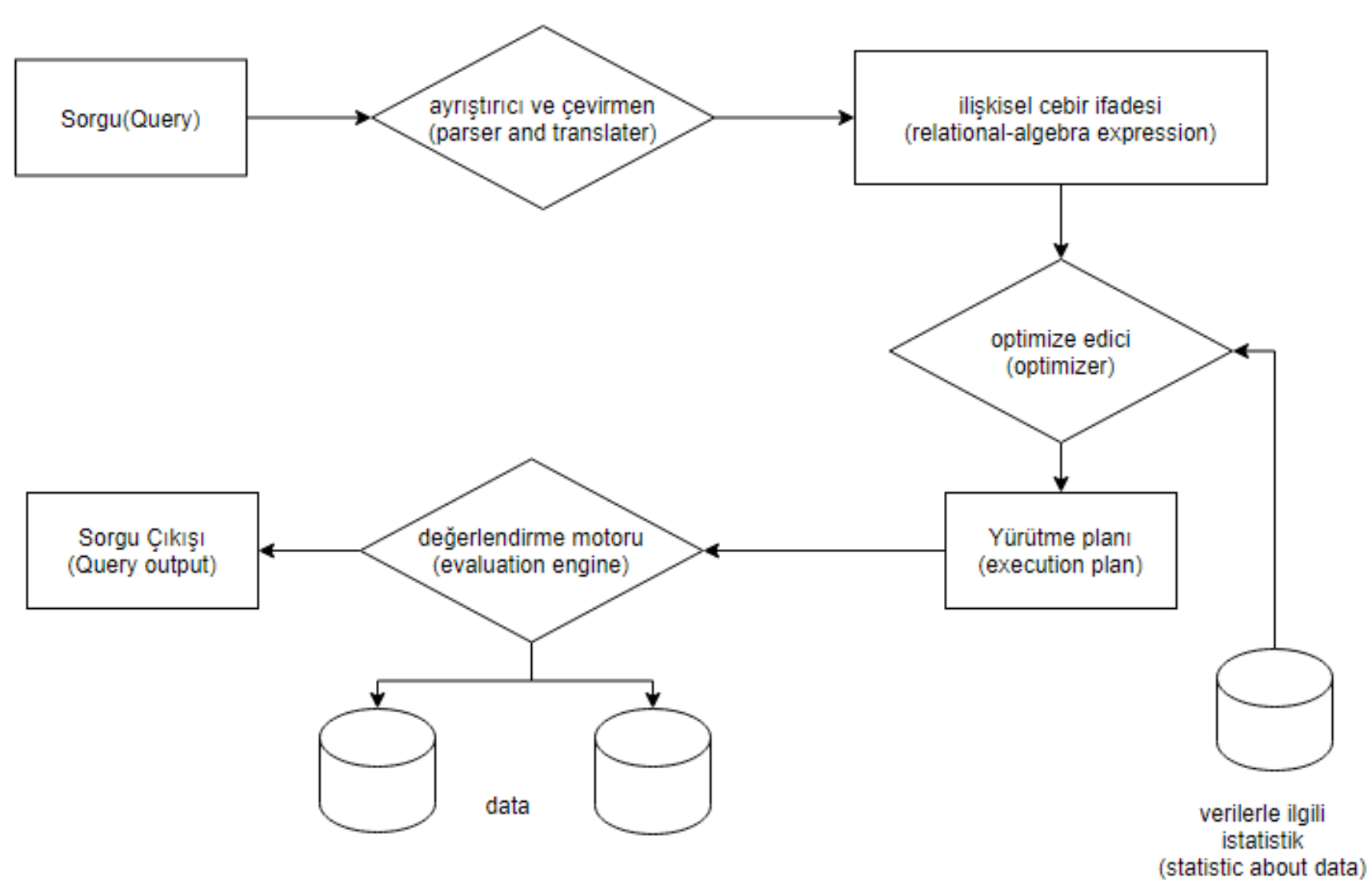
İlişkisel Cebir

- İşlemlerin tasarımında kullanılır ve SQL'in kavramsal temelini oluşturur.
- İlişkilere ve sonuç olarak verilen ilişkiye uygulanan, ancak DB'deki gerçek ilişkiyi değiştirmeyen operatörler ve manipülatörlerden oluşur.
- BBMS ile kullanılan prosedürel bir ara dildir.



DBMS'de İlişkisel Cebirin Yapısı

İlişkisel Cebir(Devam)



İlişkisel Cebir(devam)

- Avantajları:
  - Matematiksel arka plan, birçok ilginç gelişmenin ve teoremin temelidir.
  - Sorgu iyileştirici en verimli biçimi değerlendirir

İlişkisel Cebir(devam)

- İlişkisel cebir, ilişki örneklerini girdi olarak alan ve çıktı olarak ilişki örneklerini veren bir prosedürel sorgu dilidir.
- Sorgu yapmak için operatörleri kullanır.
- Bir operatör **tekli** veya **ikili olabilir** .
- İlişkileri girdi, verim ilişkilerini çıktı olarak kabul ederler.
- İlişkisel cebir, bir ilişki üzerinde yinelemeli olarak gerçekleştirilir ve ara sonuçlar da ilişkiler olarak kabul edilir.
- İlişkisel cebirin temel işlemleri:
  - Select(σ):
    - İlişkiden bir satır alt kümesi seçer
    - **Gösterim** - $\sigma_{p}(r)$
    - $\sigma_{subject = "database"}(Books)$
    - **Çıktı** - Konunun 'veritabanı' olduğu kitaplardan demetleri seçer.
    - $\sigma_{subject = "database" \text{ and } price = "450"}(Books)$

İlişkisel Cebir(devam)

- Select(σ):
  - $\sigma_{A=C}(r)$
  - σ karşılaştırma belirtimi
  - r tablonun adı olan ilişki anlamına gelir
  - p önerme mantığıdır
  - Örnek

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{A=B \wedge D > 5}(r)$

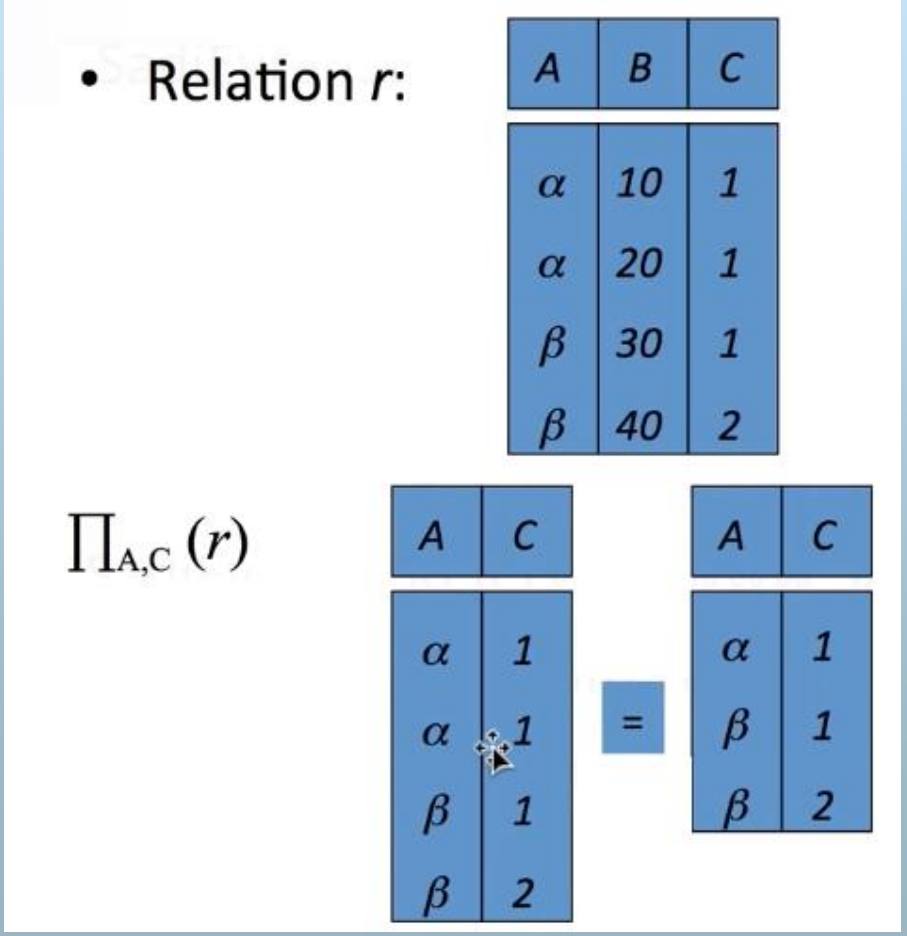
A	B	C	D
α	α	1	7
β	β	23	10

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(devam):
  - Project(π):
    - Belirli bir koşulu karşılayan sütunları yansıtır.
    - Gösterim - $\prod_{A_1, A_2, A_n} (r)$
    - Burada A_1 , A_2 , A_n , **r** ilişkisinin öznitelik isimleridir .
    - İlişki bir küme olduğundan, yinelenen satırlar otomatik olarak ortadan kaldırılır.
    - $\prod_{konu, yazar}$ (Kitaplar)
    - **Çıktı:** Kitaplar ilişkisinden konu ve yazar adlı sütunları seçer ve yansıtır.

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(devam):
  - Project(π):
  - Örnek:

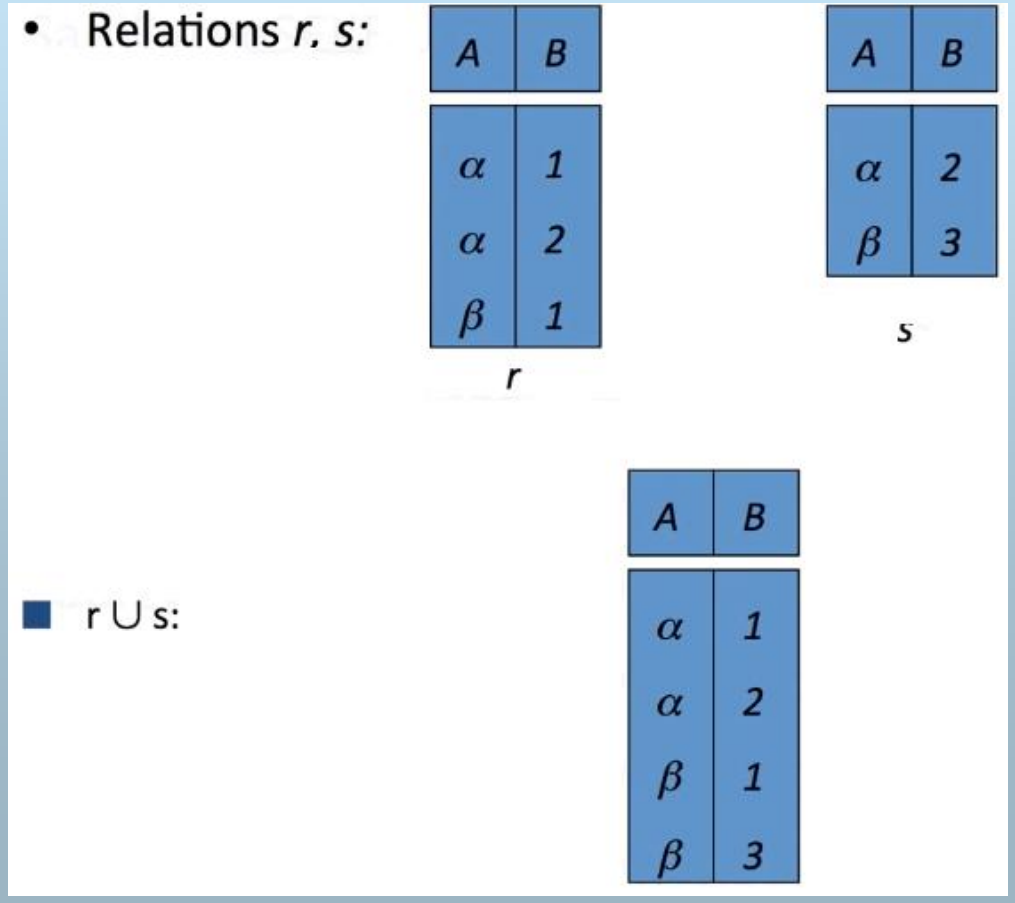


İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Union
    - Verilen iki ilişki arasında ikili birleşmeyi gerçekleştirir ve şu şekilde tanımlanır
    - $r \cup s = \{ t \mid t \in r \text{ or } t \in s\}$
    - **Gösterim** − r U s
    - Bir birleşim işleminin geçerli olması için aşağıdaki koşulların geçerli olması gerekir -
      - **r** ve **s** aynı sayıda özniteliğe sahip olmalıdır.
      - Öznitelik alanları uyumlu olmalıdır.
      - Yinelenen demetler otomatik olarak ortadan kaldırılır.
    - $\prod_{\text{author}} (\text{Books}) \cup \prod_{\text{author}} (\text{Articles})$
    - **Çıktı** - Bir kitap veya makale veya her ikisini birden yazan yazarların adlarını yansıtır.

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Union
  - Örnek



İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Set different (Farkı Ayarla)(-)
    - Küme farkı işleminin sonucu, bir ilişkide bulunan ancak ikinci ilişkide olmayan tuplelardır.
    - **Gösterim - r - s**
    - $\prod_{author}(Books) - \prod_{author}(Articles)$αβ
    - **Çıktı** - Kitap yazan ancak makale yazmayan yazarların adını verir.
  - Örnek

Relations r, s

A	B
a	1
a	2
b	1

r

A	B
a	2
b	3

s

r-s

A	B
a	1
b	1

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Kartezyen Çarpımı(X):
    - İki farklı ilişkinin bilgilerini tek bir yerde birleştirir.
    - **Gösterim** - r X s
    - r X s = {qt | q ∈ r ve t ∈ s}
    - $\sigma_{author = 'abc'}$(Books X Articles)
    - **Çıktı** - Eğitim noktası tarafından yazılan tüm kitapları ve makaleleri gösteren bir ilişki verir.

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Kartezyen Çarpımı(X):
  - Örnek

Relations r, s

A	B
a	1
b	2

r

C	D	E
a	10	a
b	10	a
b	20	b
c	10	b

s

rXs

A	B	C	D	E
a	1	a	10	a
a	1	b	10	a
a	1	b	20	b
a	1	c	10	b
b	2	a	10	a
b	2	b	10	a
b	2	b	20	b
b	2	c	10	b

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - Operatörlerin Birlikte Kullanımı:
  - Örnek

Relations r, s

A	B
a	1
b	2

r

C	D	E
a	10	a
b	10	a
b	20	b
c	10	b

s

rXs

A	B	C	D	E
a	1	a	10	a
a	1	b	10	a
a	1	b	20	b
a	1	c	10	b
b	2	a	10	a
b	2	b	10	a
b	2	b	20	b
b	2	c	10	b

$\sigma_{A=C}(rXs)$

A	B	C	D	E
a	1	a	10	a
b	2	b	10	a
b	2	b	20	b

İlişkisel Cebir(devam)

- İlişkisel cebirin temel işlemleri(Devam):
  - İşlemi Yeniden Adlandır(Rename)(ρ):
    - İlişkisel cebirin sonuçları da ilişkilerdir, ancak herhangi bir isim yoktur.
    - Yeniden adlandırma işlemi, çıktı ilişkisini yeniden adlandırmamızı sağlar
    - **Gösterim** - $\rho_x(E)$
    - **E** ifadesinin sonucunun **x** adıyla kaydedildiği yer .